UNIVERSITAT POLITÈCNICA DE CATALUNYA
BARCELONATECH
Facultat d'Informàtica de Barcelona

FIB

DEEP KERNEL NETWORKS: NEURAL ARCHITECTURES WITH SVM FOUNDATIONS

JOAN ACERO POUSA

Thesis supervisor
LUIS ANTONIO BELANCHE MUÑOZ (Department of Computer Science)

Degree
Master's Degree in Data Science

Master's thesis

Facultat d'Informàtica de Barcelona (FIB)

Universitat Politècnica de Catalunya (UPC) - BarcelonaTech

Abstract

Key Words:

Acknowledgments

I would like to express my sincere gratitude to Prof. Luis Antonio Belanche Muñoz, my Bachelor's and Master's thesis supervisor, as well as my professor, for giving me the opportunity to learn from him and research under his guidance. His insightful advice and experience have been fundamental to my development as a researcher and have contributed to the completion of this thesis.

I would also like to express my appreciation to the other professors who have taught me throughout my Bachelor's and Master's studies. Their knowledge has greatly contributed to my academic growth.

Acronyms

ANN Artificial Neural Network. 9, 11, 23–30, 32, 34, 35, 54, 55, 59

DCKM Deep Convex Kernel Machine. 34, 36–39

RBF Radial Basis Function. 44–46, 54–56, 60, 69

ReLU Rectified Linear Unit. 45, 60

RFF Random Fourier Features. 20, 31, 36, 37, 54, 56, 60

SGD Stochastic Gradient Descent. 60

SVC Support Vector Classifier. 6, 12, 13, 15–17, 27

SVM Support Vector Machine. 6, 9–12, 15, 18–20, 23, 26–28, 30–54, 56, 60, 70, 71

SVR Support Vector Regressor. 20–23

UCI University of California, Irvine. 42, 43

Contents

1 Introduction

Support Vector Machines (SVMs) are powerful machine learning models with a strong mathematical foundation. They are versatile due to the kernel trick, include inherent regularisation properties [19], and have been shown to be highly effective in both classification and regression tasks [23]. Their optimisation problem is convex, so training is free of the local minima and initialisation sensitivity that affect Artificial Neural Networks (ANNs).

This strength, however, does not extend to two increasingly common settings. First, kernel SVMs scale poorly: they require the construction of an $n \times n$ kernel matrix (where n is the number of samples), incurring a cost of $O(n^2)$ in memory and between $O(n^2)$ and $O(n^3)$ in time, which becomes prohibitive for large n [16]. Second, SVMs are shallow models: they apply a single fixed feature transformation, and therefore cannot learn the hierarchical, multi-level representations that underlie the success of deep architectures [38]. While kernel approximation techniques address the first limitation in isolation, doing so while also introducing depth, without sacrificing the convexity that makes SVMs attractive, remains an open problem.

1.1 Motivation

This work is motivated by the goal of extending the strong performance of SVMs to precisely those settings in which they are traditionally less effective: large-scale datasets, where exact kernel methods are computationally infeasible, and highly complex data, where a shallow decision function is insufficient.

We aim to address both limitations simultaneously within a single architecture, while retaining the convex, margin-based optimisation that distinguishes SVMs from ANNs.

1.2 Objectives

The main objective of this thesis is to introduce a novel deep architecture that integrates SVMs with the hierarchical feature extraction characteristics of ANNs, improving the scalability and performance of support vector methods on large-scale and highly complex datasets. More specifically, our objectives are to:

- Extend the SVM framework to a multi-layer architecture capable of hierarchical feature learning.
- Address the scalability limitations inherent in kernel SVMs through an explicit feature-map approximation.

- Develop a training algorithm that avoids backpropagation and preserves the convex optimisation advantages of SVMs.
- Analyse the computational complexity of the proposed method.
- Validate each architectural design decision through an extensive empirical evaluation.
- Benchmark the proposed method against state-of-the-art alternatives.

1.3 Thesis Structure

The thesis is organised as follows. Section 2 provides the foundational knowledge required to understand the subsequent discussions. Section 3 reviews the state-of-the-art literature, establishing the starting point for our investigation. In Section 4, we describe the derivation process of the proposed architecture, including its training algorithm and a computational evaluation. Section 5 presents a complete set of experimental work together with a discussion of the findings. Finally, Section 6 summarises the conclusions of this work and proposes potential paths for future research.

The source code and implementation details to reproduce the experimental results are publicly available in this GitHub repository.

2 Background

This chapter presents the theoretical foundations required for the development of the proposed architecture. Section 2.1 presents the supervised learning framework. Section 2.2 describes the mathematical formulation of SVMs, including margin optimisation, duality, the kernel trick, and its approximation through random features. Section 2.3 reviews the architecture and training mechanisms of ANNs. Finally, the chapter connects these methodologies by representing SVMs as single-layer networks and introducing the arc-cosine kernel, which formally links kernel methods to infinite-width neural layers.

2.1 Supervised Learning

Supervised learning is a fundamental setting in machine learning where the algorithm learns from labelled data to generalise predictions for new, unseen data [9]. This section describes the primary supervised learning tasks addressed by the proposed architecture: classification and regression.

2.1.1 Classification

Classification involves assigning a given input to one of several predefined categories or classes. Formally, given a dataset $D = \{(\mathbf{x}_i, y_i)\}_{i=1}^n$, where $\mathbf{x}_i \in \mathbb{R}^d$ represents the feature vector of the i-th instance and $y_i \in \{1, 2, \ldots, K\}$ is the corresponding class label, the goal is to learn a function $f : \mathbb{R}^d \to \{1, 2, \ldots, K\}$ that minimises the discrepancy between the predicted labels $\hat{y}_i = f(\mathbf{x}_i)$ and the actual labels y_i. The process includes a training phase, where the best mapping function f^* is learned, and a prediction phase, where the trained model is used to classify new inputs. The performance is typically evaluated using a loss function $L(y_i, \hat{y}_i)$, which measures the cost of incorrect predictions.

2.1.2 Regression

Regression aims to predict a continuous output value for a given input. Formally, given a dataset $D = \{(\mathbf{x}_i, y_i)\}_{i=1}^n$, where $\mathbf{x}_i \in \mathbb{R}^d$ is the feature vector of the i-th instance and $y_i \in \mathbb{R}$ is the continuous output value, the objective is to learn a function $f : \mathbb{R}^d \to \mathbb{R}$ that minimises the difference between the predicted values $\hat{y}_i = f(\mathbf{x}_i)$ and the actual values y_i. Like classification, the process involves a training phase to learn the best hypothesis f^* and a prediction phase to estimate continuous values for new data. The effectiveness is assessed using a loss function $L(y_i, \hat{y}_i)$, which quantifies the error of the predictions.

2.2 Support Vector Machines

The theoretical foundations of SVMs can be traced back to the development of statistical learning theory by Vapnik and Chervonenkis [59]. Building upon this framework, Boser, Guyon, and Vapnik formally introduced the optimal margin classifier in [13]. Subsequently, Cortes and Vapnik extended the method to handle non-separable datasets through the introduction of the soft-margin formulation [19], significantly broadening its applicability to real-world problems.

The following sections provide an explanation of the main principles of SVMs, followed by their advanced extensions. The explanation is inspired by [9] and [19].

We first provide the mathematical formulation of the linear SVM optimisation problem for classification. We then extend the explanation to the non-linear case in Section 2.2.2. Finally, we present the adaptation for regression tasks in Section 2.2.3.

2.2.1 Linear Support Vector Classification

Support Vector Classifiers (SVCs) approach the classification problem using the concept of margin, which is described as the smallest distance between the hyperplane and any of the samples. SVCs find the hyperplane that maximises the margin. In a two-class classification problem, the training data set D contains n input vectors $\mathbf{x}_1, \ldots, \mathbf{x}_n$ and their respective labels $y_1, \ldots, y_n$, where $y_i \in \{-1, 1\}$. A hyperplane found in a d dimensional feature space is expressed mathematically as:

$$h(\mathbf{x}) = \mathbf{w}^T\mathbf{x} + b \tag{1}$$

where $\mathbf{w} \in \mathbb{R}^d$ is the weight vector and $b \in \mathbb{R}$ is the bias term. During the testing phase, the SVC classifies new observations $\mathbf{x}$ according to $\text{sign}(h(\mathbf{x}))$.

We define the hyperplane that meets $h(\mathbf{x}) = 0$ as follows:

$$H = \{\mathbf{x} \in \mathbb{R}^d \mid \mathbf{w}^T\mathbf{x} + b = 0\} \tag{2}$$

Note that samples which are closest to H are known as support vectors and are responsible for defining the margin.

In Figure 1, a dataset is represented with blue circles representing points of one class and red triangles representing points of the other. The optimised hyperplane, denoted as H, is portrayed as a bold line. The margin is indicated as M. Support vectors are uncoloured data points positioned precisely on the dashed lines.

Hard-margin SVC We first describe the initial SVC proposal, which assumes data is perfectly separable.

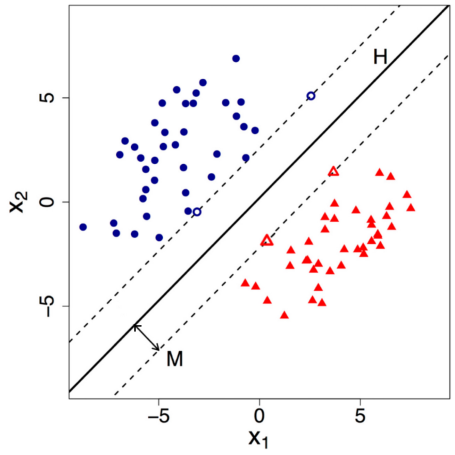

Figure 1: Two-class classification problem in a two-dimensional feature space solved by a hard-margin SVC. Figure taken from [60].

The perpendicular distance from a general point $\mathbf{x}$ to the hyperplane H, given by $h(\mathbf{x})$ defined in Equation 1, is given by (see Figure 4.1 in [9]):

$$\frac{|h(\mathbf{x})|}{\|\mathbf{w}\|} = \frac{|\mathbf{w}^T\mathbf{x} + b|}{\|\mathbf{w}\|} \tag{3}$$

Moreover, since hard-margin SVCs assume the data is linearly separable and focus on solutions without misclassification, we know that a feasible $\mathbf{w}, b$ satisfies $y_i h(\mathbf{x}_i) > 0 \ \forall i$. Thus, the distance from a point $\mathbf{x}_i$ to H is defined as:

$$\frac{y_i h(\mathbf{x}_i)}{\|\mathbf{w}\|} = \frac{y_i(\mathbf{w}^T\mathbf{x}_i + b)}{\|\mathbf{w}\|} \tag{4}$$

Now, suppose we rescale the weight vector $\mathbf{w}$ and the bias b by κ: $\mathbf{w} \to \kappa\mathbf{w}$ and $b \to \kappa b$. Note that the distance from $\mathbf{x}_i$ to the decision surface after rescaling remains unchanged:

$$\frac{(\kappa\mathbf{w}^T\mathbf{x}_i + \kappa b)}{\|\kappa\mathbf{w}\|} = \frac{\kappa(\mathbf{w}^T\mathbf{x}_i + b)}{\|\kappa\mathbf{w}\|} = \frac{\kappa(\mathbf{w}^T\mathbf{x}_i + b)}{\kappa\|\mathbf{w}\|} = \frac{\mathbf{w}^T\mathbf{x}_i + b}{\|\mathbf{w}\|} \tag{5}$$

Therefore, we can rescale the margin boundaries to exactly distance 1 for the support vectors:

$$y_i(\mathbf{w}^T\mathbf{x}_i + b) = 1 \quad \forall \mathbf{x}_i \in B \tag{6}$$

where B is the set of support vectors. Since data points of each class must be on or beyond the margin defined by the support vectors, all data points satisfy:

$$y_i(\mathbf{w}^T\mathbf{x}_i + b) \geq 1 \quad i = 1, \ldots, n \tag{7}$$

As previously stated, SVCs seek the specific hyperplane that maximises the margin

between classes. Using Equations 6 and 4, we can derive the mathematical expression of the margin (often referred to as the half-margin, representing the distance from the hyperplane to the support vectors):

$$M = \frac{1}{\|\mathbf{w}\|} \tag{8}$$

To maximise the margin, which is equivalent to minimising $\|\mathbf{w}\|$, we can define the optimisation problem as follows:

$$\min_{\mathbf{w},b} \frac{1}{2}\|\mathbf{w}\|^2 \tag{9}$$

$$\text{subject to} \quad y_i(\mathbf{w}^T\mathbf{x}_i + b) \geq 1 \quad \forall i \tag{10}$$

Here, the factor $\frac{1}{2}$ and the squared term are included for mathematical convenience, simplifying the derivative during optimisation. This expression is known as the primal problem.

To solve the primal problem, we use the method of Lagrange multipliers [37]. We construct the Lagrangian function as:

$$\mathcal{L}(\mathbf{w}, b, \boldsymbol{\alpha}) = \frac{1}{2}\|\mathbf{w}\|^2 - \sum_{i=1}^{n} \alpha_i[y_i(\mathbf{w}^T\mathbf{x}_i + b) - 1] \tag{11}$$

where $\alpha_i \geq 0$ are the Lagrange multipliers and n is the number of data points.

To find the optimal solution, we take the partial derivatives of $\mathcal{L}$ with respect to $\mathbf{w}$ and b and set them to zero:

$$\frac{\partial \mathcal{L}}{\partial \mathbf{w}} = \mathbf{w} - \sum_{i=1}^{n} \alpha_i y_i \mathbf{x}_i = 0 \tag{12}$$

$$\frac{\partial \mathcal{L}}{\partial b} = -\sum_{i=1}^{n} \alpha_i y_i = 0 \tag{13}$$

From these conditions, we get:

$$\mathbf{w} = \sum_{i=1}^{n} \alpha_i y_i \mathbf{x}_i \tag{14}$$

$$\sum_{i=1}^{n} \alpha_i y_i = 0 \tag{15}$$

Substituting $\mathbf{w}$ back into the Lagrangian, we obtain the dual form:

$$\mathcal{L}_D(\boldsymbol{\alpha}) = \sum_{i=1}^{n} \alpha_i - \frac{1}{2} \sum_{i=1}^{n} \sum_{j=1}^{n} \alpha_i \alpha_j y_i y_j \mathbf{x}_i^T \mathbf{x}_j \tag{16}$$

Hence, the dual optimisation problem is:

$$\max_{\boldsymbol{\alpha}} \sum_{i=1}^{n} \alpha_i - \frac{1}{2} \sum_{i=1}^{n} \sum_{j=1}^{n} \alpha_i \alpha_j y_i y_j \mathbf{x}_i^T \mathbf{x}_j \tag{17}$$

$$\text{subject to} \quad \sum_{i=1}^{n} \alpha_i y_i = 0 \quad \text{and} \quad \alpha_i \geq 0 \quad \forall i \tag{18}$$

After optimising the model, the SVC classifies new data points evaluating $\text{sign}(h(\mathbf{x}))$, defined in Equation 1. The hyperplane function can also be expressed as (using Equation 14):

$$h(\mathbf{x}) = \sum_{i=1}^{n} \alpha_i y_i \mathbf{x}^T \mathbf{x}_i + b \tag{19}$$

Note that this constrained optimisation problem satisfies the Karush-Kuhn-Tucker conditions [35], which require that the following constraints hold for $i = 1, \ldots, n$:

$$\alpha_i \geq 0 \tag{20}$$

$$y_i h(\mathbf{x}_i) - 1 \geq 0 \tag{21}$$

$$\alpha_i (y_i h(\mathbf{x}_i) - 1) = 0 \tag{22}$$

From these constraints, we extract that for every data point, either $\alpha_i = 0$ or $y_i h(\mathbf{x}_i) = 1$. Thus, only support vectors have $\alpha_i > 0$, since they satisfy $y_i h(\mathbf{x}_i) = 1$. Therefore, the decision function of the SVC (Equation 19) can be redefined for doing the summation over support vectors instead of all data points. This is a crucial characteristic regarding the practical applicability of SVMs since when the model is trained, a significant proportion of the data points can be discarded.

Soft-margin SVC While the hard-margin SVC formulation assumes perfect data separability, this is often incorrect in real-world scenarios. Noise, outliers, or simply intrinsic overlap between classes can cause the hard margin approach to be impractical. Soft-margin SVCs solve this problem by allowing for some misclassification, thereby increasing the model's robustness and generalisation capability.

In soft-margin SVCs, the optimisation objective balances two competing goals: maximising the margin between classes and minimising the classification error. Moreover, this approach introduces an inherent form of regularisation in its formulation, making SVMs less prone to overfitting.

See Figure 2 for a visual reference of the soft-margin SVC. Data points are represented by blue circles and red triangles, one representing each class. One can see that data points are allowed to be misclassified or inside the margin area.

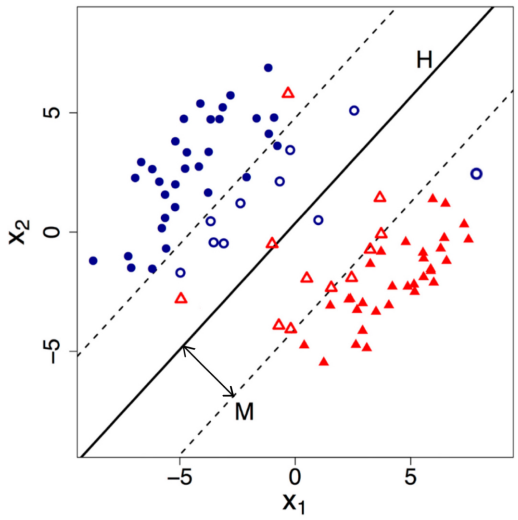

Figure 2: Two-class classification problem in a two-dimensional feature space solved by a soft-margin SVC. Figure taken from [60].

The mentioned capabilities are achieved by introducing slack variables. Slack variables, denoted by $\xi_i \geq 0$, represent the deviation of data points from being correctly classified according to the decision boundary. Each slack variable ξ_i corresponds to a specific data point $\mathbf{x}_i$. If $\xi_i = 0$, it indicates that the point $\mathbf{x}_i$ is correctly classified and lies on the correct side of the margin boundary or over H. If $\xi_i > 0$, it symbolises that the point $\mathbf{x}_i$ is misclassified or lies on the wrong side of the margin boundary, with a magnitude indicating the extent of misclassification.

The optimisation problem is then modified to minimise both the margin size and the sum of these slack variables:

$$\min_{\mathbf{w},b,\boldsymbol{\xi}} \frac{1}{2}\|\mathbf{w}\|^2 + C\sum_{i=1}^{n}\xi_i \tag{23}$$

$$\text{subject to} \quad y_i(\mathbf{w}^T\mathbf{x}_i + b) \geq 1 - \xi_i \quad \forall i \tag{24}$$

$$\xi_i \geq 0 \quad \forall i \tag{25}$$

Here, $C > 0$ is a hyperparameter that controls the trade-off between maximising the margin and minimising the classification error. A larger value of C implies a higher penalty for misclassification, leading to a tighter decision boundary.

To solve this optimisation problem, we again employ the Lagrange multipliers

method [37]. The Lagrangian function for the soft-margin SVC is given by:

$$\mathcal{L}(\mathbf{w}, b, \boldsymbol{\xi}, \boldsymbol{\alpha}, \boldsymbol{\mu}) = \frac{1}{2}\|\mathbf{w}\|^2 + C\sum_{i=1}^{n}\xi_i - \sum_{i=1}^{n}\alpha_i\left[y_i(\mathbf{w}^T\mathbf{x}_i + b) - 1 + \xi_i\right] - \sum_{i=1}^{n}\mu_i\xi_i \quad (26)$$

Where $\boldsymbol{\alpha} \geq 0$ and $\boldsymbol{\mu} \geq 0$ are the Lagrange multipliers associated with the inequality and slackness constraints, respectively.

We then take the partial derivatives of $\mathcal{L}$ with respect to $\mathbf{w}$, b, and $\boldsymbol{\xi}$, and set them to zero to find the optimal solution:

$$\frac{\partial\mathcal{L}}{\partial\mathbf{w}} = \mathbf{w} - \sum_{i=1}^{n}\alpha_i y_i\mathbf{x}_i = 0 \Rightarrow \mathbf{w} = \sum_{i=1}^{n}\alpha_i y_i\mathbf{x}_i \quad (27)$$

$$\frac{\partial\mathcal{L}}{\partial b} = -\sum_{i=1}^{n}\alpha_i y_i = 0 \quad (28)$$

$$\frac{\partial\mathcal{L}}{\partial\xi_i} = C - \alpha_i - \mu_i = 0 \Rightarrow \alpha_i = C - \mu_i \quad (29)$$

Substituting these results back into the Lagrangian, we obtain the dual problem:

$$\mathcal{L}_D(\boldsymbol{\alpha}) = \sum_{i=1}^{n}\alpha_i - \frac{1}{2}\sum_{i=1}^{n}\sum_{j=1}^{n}\alpha_i\alpha_j y_i y_j \mathbf{x}_i^T\mathbf{x}_j \quad (30)$$

subject to the constraints:

$$0 \leq \alpha_i \leq C, \quad (31)$$

$$\sum_{i=1}^{n}\alpha_i y_i = 0 \quad (32)$$

Once the dual problem is solved, the optimal hyperplane parameters $\mathbf{w}$ and b, along with the slack variables ξ_i, can be obtained. To classify a new data point $\mathbf{x}$, the SVC uses $\text{sign}(h(\mathbf{x}))$, defined in Equation 19.

As stated, the soft-margin SVC aims to balance maximising the margin and minimising classification errors. The hinge loss is used to achieve this balance in the training phase. The hinge loss function is defined as:

$$L(y_i, h(\mathbf{x}_i)) = \max(0, 1 - y_i h(\mathbf{x}_i)) \quad (33)$$

The hinge loss penalises misclassified points and points within the margin, with ξ_i representing the amount by which the constraint $y_i(\mathbf{w}^T\mathbf{x}_i + b) \geq 1$ is violated.

2.2.2 Non-Linear Classification and the Kernel Trick

Many real-world problems involve datasets that exhibit non-linear relationships, rendering linear SVMs insufficient for practical prediction. This limitation underscores the need for non-linear SVMs.

For managing non-linearly separable data, SVMs can explicitly map the input data into a higher-dimensional space using a function [13]:

$$\phi : \mathbb{R}^d \to \mathbb{R}^m \quad \text{with} \quad m \geq d$$

where the ϕ function is generally defined to map data from the original space to a (possibly infinite-dimensional) Hilbert space [67], so that m may be infinite. This mapping allows the SVM to construct a linear decision boundary in the higher-dimensional space, corresponding to a non-linear boundary in the original space (see Figure 3). Thus, the SVM dual optimisation problem (see Equation 30) can be rewritten by replacing the dot product between samples in the original feature space with the inner product of the samples in a high dimensional space. In the classification case, the resulting dual problem is, with the constraint formulation unchanged:

$$\mathcal{L}_D(\boldsymbol{\alpha}) = \sum_{i=1}^{n} \alpha_i - \frac{1}{2} \sum_{i=1}^{n} \sum_{j=1}^{n} \alpha_i \alpha_j y_i y_j \langle \phi(\mathbf{x}_i), \phi(\mathbf{x}_j) \rangle \tag{34}$$

The operation $\langle \phi(\mathbf{x}_i), \phi(\mathbf{x}_j) \rangle$ involves mapping the original vectors $\mathbf{x}_i$, $\mathbf{x}_j$ to the high dimensional space, and subsequently computing the inner product between them.

Consider the following example illustrating the mapping to a space for a polynomial kernel of degree 2. The steps involved are:

1. Original vectors: $\mathbf{x} = (x_1, x_2)$, $\mathbf{y} = (y_1, y_2)$
2. Mapped vectors: $\phi(\mathbf{x}) = (x_1^2, \sqrt{2}x_1x_2, x_2^2)$, $\phi(\mathbf{y}) = (y_1^2, \sqrt{2}y_1y_2, y_2^2)$
3. Inner product in the higher-dimensional space: $\langle \phi(\mathbf{x}), \phi(\mathbf{y}) \rangle = x_1^2y_1^2 + 2x_1x_2y_1y_2 + x_2^2y_2^2 = (\mathbf{x}^T\mathbf{y})^2$

The Kernel Trick The kernel trick was introduced in [13]. Using it, inner products in high-dimensional feature spaces can be efficiently computed, avoiding the computational disadvantage of explicit mapping.

Inner products, known as dot products in the context of Euclidean space, are a fundamental operation in linear algebra. For two vectors $\mathbf{u}$ and $\mathbf{v}$ in a vector space, the inner product $\langle \mathbf{u}, \mathbf{v} \rangle$ is a scalar value [27].

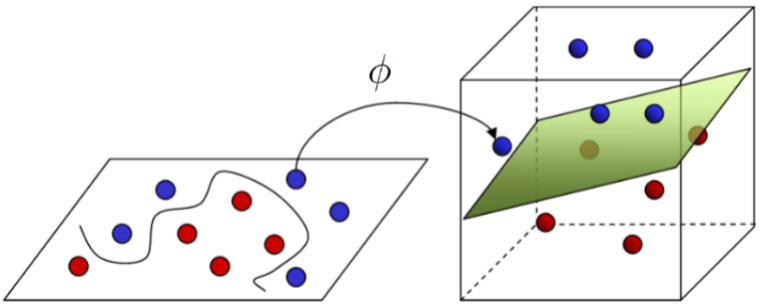

Figure 3: Mapping ϕ from a two-dimensional to a three-dimensional feature space, where data is linearly separable. Image taken from [28].

The main idea is to use a kernel function $K(\mathbf{x}_i, \mathbf{x}_j)$ to compute the inner product in the transformed space instead of explicitly computing $\langle \phi(\mathbf{x}_i), \phi(\mathbf{x}_j) \rangle$:

$$K(\mathbf{x}_i, \mathbf{x}_j) = \langle \phi(\mathbf{x}_i), \phi(\mathbf{x}_j) \rangle \tag{35}$$

By substituting the inner product with a kernel function, the SVM optimisation problem is again rewritten. In the classification case, the resulting dual problem is, with the constraints unchanged:

$$\mathcal{L}_D(\boldsymbol{\alpha}) = \sum_{i=1}^{n} \alpha_i - \frac{1}{2} \sum_{i=1}^{n} \sum_{j=1}^{n} \alpha_i \alpha_j y_i y_j K(\mathbf{x}_i, \mathbf{x}_j) \tag{36}$$

We now give the definition of the Kernel functions that we use in subsequent chapters. Each of these functions implicitly computes inner products in different high-dimensional spaces:

- **Polynomial Kernel:** $K(\mathbf{x}_i, \mathbf{x}_j) = (\mathbf{x}_i^T \mathbf{x}_j + c)^d$, where c is a constant and d is the degree of the polynomial.
- **Radial Basis Function (RBF) Kernel:** $K(\mathbf{x}_i, \mathbf{x}_j) = \exp\left(-\frac{\|\mathbf{x}_i - \mathbf{x}_j\|^2}{2\sigma^2}\right)$, where σ defines the width of the Gaussian function. It is powerful for data with complex relationships, creating flexible decision boundaries.
- **Laplacian Kernel:** $K(\mathbf{x}_i, \mathbf{x}_j) = \exp\left(-\frac{\|\mathbf{x}_i - \mathbf{x}_j\|}{\sigma}\right)$, where σ controls the kernel width. It is similar to the RBF kernel but uses the L_1 distance.
- **Cauchy Kernel:** $K(\mathbf{x}_i, \mathbf{x}_j) = \frac{1}{1 + \frac{\|\mathbf{x}_i - \mathbf{x}_j\|^2}{2\sigma^2}}$, where σ influences the kernel's width.

By selecting an appropriate kernel function, SVMs can effectively capture and model non-linear structures, enabling the fitting of complex data relationships. However, note that using kernel functions requires the computation of a matrix of inner

products between all data points. This kernel matrix has n^2 entries and requires $O(n^2 d)$ operations to build and between $O(n^2)$ and $O(n^3)$ operations to solve, so both its memory and time costs grow at least quadratically with the number of samples n. As already mentioned, this computation becomes infeasible for large-scale problems [16].

Shift-Invariant Kernels Shift-invariant kernels are a particular class of kernel functions where the value of the kernel depends only on the difference between the input vectors $(x_i - x_j)$ [11]. This property is beneficial in scenarios where the location of features in the input space is not as important as their relative positions. Examples of shift-invariant kernels include the RBF, Laplacian and Cauchy kernels.

Random Feature Approximations As noted above, the kernel trick avoids constructing the feature map ϕ explicitly, but at the cost of forming the full $n \times n$ kernel matrix, whose quadratic growth in the number of samples n makes it intractable for large-scale problems. Random feature methods [51] address this limitation from the opposite direction: instead of computing inner products implicitly through a kernel, they construct an *explicit*, finite-dimensional feature map $\mathbf{z} : \mathbb{R}^d \to \mathbb{R}^P$ such that the inner product in this low-dimensional space approximates the kernel,

$$K(\mathbf{x}_i, \mathbf{x}_j) \approx \mathbf{z}(\mathbf{x}_i)^T \mathbf{z}(\mathbf{x}_j), \tag{37}$$

where P is the number of random features. Once the inputs are mapped through $\mathbf{z}$, a *linear* model can be trained directly on the transformed data, recovering a non-linear decision boundary in the original space while avoiding the kernel matrix entirely. This reduces the cost from the quadratic $O(n^2)$ of the kernel matrix to $O(nP)$, which is linear in the number of samples and therefore suitable for large-scale settings.

For shift-invariant kernels, Bochner's theorem [11] guarantees that a continuous shift-invariant kernel is positive-definite if and only if it is the Fourier transform of a non-negative measure. Rahimi and Recht [51] exploited this result to approximate such kernels by sampling frequencies from the corresponding measure, yielding the well-known Random Fourier Featuress (RFFs) for the RBF kernel. More broadly, whenever a kernel can be expressed as an expectation over a known distribution, an explicit feature map of this kind can be constructed by Monte Carlo sampling from that distribution.

2.2.3 Support Vector Regression

In regression tasks, SVMs can be adapted to predict continuous values rather than discrete class labels. Similar to classification tasks, variations and extensions address different aspects or challenges in regression problems. We present the standard Support Vector Regressors (SVRs) approach.

In this setting, SVRs aim to fit a function $f(\mathbf{x}) = \mathbf{w}^T\mathbf{x} + b$ that has at most ϵ deviation from the actual target y_i for all training data points $\mathbf{x}_i$. This is known as the ϵ-insensitive loss function.

The primal optimisation problem for regression can be formulated as follows:

$$\min_{\mathbf{w},b,\boldsymbol{\xi},\boldsymbol{\xi}^*} \frac{1}{2}\|\mathbf{w}\|^2 + C\sum_{i=1}^{n}(\xi_i + \xi_i^*) \tag{38}$$

$$\text{subject to} \quad \begin{cases} y_i - (\mathbf{w}^T\mathbf{x}_i + b) \leq \epsilon + \xi_i \\ (\mathbf{w}^T\mathbf{x}_i + b) - y_i \leq \epsilon + \xi_i^* \\ \xi_i, \xi_i^* \geq 0 \end{cases} \tag{39}$$

Here, ξ_i and ξ_i^* are slack variables that represent deviations above and below the ϵ-tube, respectively. The ϵ-tube is a critical concept in SVRs, defining a margin of tolerance within which no penalty is assigned to errors. The parameter C controls the trade-off between the model complexity and the extent to which deviations larger than ϵ are tolerated.

Refer to Figure 4 for a graphical depiction of these introduced concepts. In this visual representation, data points are illustrated as distinctive red squares. The area between the dashed lines represents the ϵ-tube. Data points residing within this ϵ-tube do not incur any penalisation on the model, as they lie within an acceptable deviation range from the predicted function. These points, therefore, do not contribute to the loss function. Conversely, data points outside the ϵ-tube boundaries are penalised proportionally to their distance from the tube. Slack variables quantify the deviation of these points from the expected margin, denoted as ξ_i and ξ_i^*. Specifically, ξ_i represents deviations above the ϵ-tube, while ξ_i^* represents deviations below it.

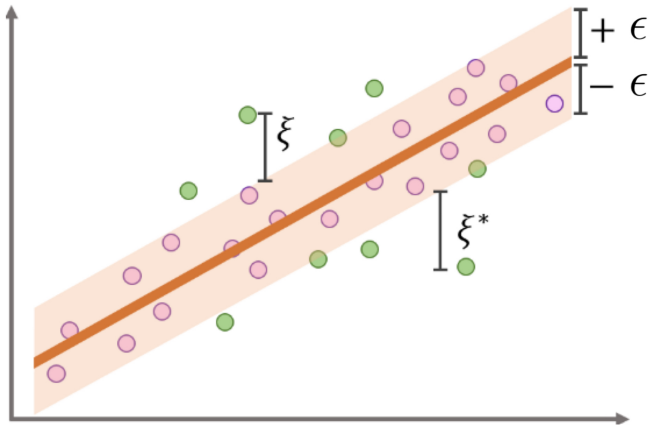

Figure 4: SVRs solving a regression task. Image from [41].

To solve the SVRs primal optimisation problem, we again seek the dual of the problem using Lagrange multipliers [37]. The Lagrangian for the regression problem is given by:

$$\begin{aligned}\mathcal{L}(\mathbf{w}, b, \boldsymbol{\xi}, \boldsymbol{\xi}^*, \boldsymbol{\mu}, \boldsymbol{\mu}^*, \boldsymbol{\alpha}, \boldsymbol{\alpha}^*) = \frac{1}{2}\|\mathbf{w}\|^2 + C\sum_{i=1}^{n}(\xi_i + \xi_i^*) - \sum_{i=1}^{n}(\mu_i\xi_i + \mu_i^*\xi_i^*) \\ - \sum_{i=1}^{n}\alpha_i(\epsilon + \xi_i - y_i + \mathbf{w}^T\mathbf{x}_i + b) - \sum_{i=1}^{n}\alpha_i^*(\epsilon + \xi_i^* - \mathbf{w}^T\mathbf{x}_i - b + y_i)\end{aligned} \tag{40}$$

where $\boldsymbol{\mu}$, $\boldsymbol{\mu}^*$, $\boldsymbol{\alpha}$, and $\boldsymbol{\alpha}^*$ are the Lagrange multipliers.

Taking the partial derivatives of $\mathcal{L}$ with respect to $\mathbf{w}$, b, ξ_i, and ξ_i^*, and setting them to zero, we get:

$$\frac{\partial\mathcal{L}}{\partial\mathbf{w}} = \mathbf{w} - \sum_{i=1}^{n}(\alpha_i - \alpha_i^*)\mathbf{x}_i = 0 \Rightarrow \mathbf{w} = \sum_{i=1}^{n}(\alpha_i - \alpha_i^*)\mathbf{x}_i \tag{41}$$

$$\frac{\partial\mathcal{L}}{\partial b} = \sum_{i=1}^{n}(\alpha_i^* - \alpha_i) = 0 \tag{42}$$

$$\frac{\partial\mathcal{L}}{\partial\xi_i} = C - \mu_i - \alpha_i = 0 \Rightarrow \mu_i = C - \alpha_i \tag{43}$$

$$\frac{\partial\mathcal{L}}{\partial\xi_i^*} = C - \mu_i^* - \alpha_i^* = 0 \Rightarrow \mu_i^* = C - \alpha_i^* \tag{44}$$

Substituting these results back into the Lagrangian, we obtain the dual problem:

$$\mathcal{L}_D(\boldsymbol{\alpha}, \boldsymbol{\alpha}^*) = \sum_{i=1}^{n} y_i(\alpha_i - \alpha_i^*) - \epsilon\sum_{i=1}^{n}(\alpha_i + \alpha_i^*) - \frac{1}{2}\sum_{i=1}^{n}\sum_{j=1}^{n}(\alpha_i - \alpha_i^*)(\alpha_j - \alpha_j^*)\mathbf{x}_i^T\mathbf{x}_j \tag{45}$$

Subject to the constraints:

$$0 \leq \alpha_i \leq C, \tag{46}$$

$$0 \leq \alpha_i^* \leq C, \tag{47}$$

$$\sum_{i=1}^{n}(\alpha_i - \alpha_i^*) = 0 \tag{48}$$

Solving the dual problem provides the values of α_i and α_i^*, from which we can determine the weight vector $\mathbf{w}$ and the bias term b. Finally, the decision function for

SVRs can be expressed as:

$$f(\mathbf{x}) = \mathbf{w}^T\mathbf{x} + b = \sum_{i=1}^{n}(\alpha_i - \alpha_i^*)\mathbf{x}_i^T\mathbf{x} + b \tag{49}$$

where $\mathbf{w}$ is recovered from the dual variables as $\mathbf{w} = \sum_{i=1}^{n}(\alpha_i - \alpha_i^*)\mathbf{x}_i$, and b is obtained from the Karush-Kuhn-Tucker conditions on the points lying on the boundary of the ϵ-tube.

As in the classification case, the dual objective and the decision function depend on the inputs only through inner products $\mathbf{x}_i^T\mathbf{x}_j$, so the kernel substitution of Section 2.2.2 applies identically to regression. Replacing each inner product with a kernel function $K(\mathbf{x}_i, \mathbf{x}_j)$ yields the kernelised dual problem:

$$\mathcal{L}_D(\boldsymbol{\alpha}, \boldsymbol{\alpha}^*) = \sum_{i=1}^{n} y_i(\alpha_i - \alpha_i^*) - \epsilon\sum_{i=1}^{n}(\alpha_i + \alpha_i^*) - \frac{1}{2}\sum_{i=1}^{n}\sum_{j=1}^{n}(\alpha_i - \alpha_i^*)(\alpha_j - \alpha_j^*)K(\mathbf{x}_i, \mathbf{x}_j) \tag{50}$$

subject to the same constraints, and the corresponding decision function:

$$f(\mathbf{x}) = \sum_{i=1}^{n}(\alpha_i - \alpha_i^*)K(\mathbf{x}_i, \mathbf{x}) + b. \tag{51}$$

2.3 Artificial Neural Networks

This section presents the essential knowledge of ANNs required for comprehending the thesis contents. Readers are encouraged to consult the provided references for a more detailed exploration of ANNs [26, 9]. In Section 2.3.6, we connect SVMs with ANNs by defining an SVM as a single hidden-layer network.

The conceptualisation of ANNs began with the foundational computational model of the neuron proposed by McCulloch and Pitts [44]. Rosenblatt [53] later presented the perceptron, establishing a framework for binary classification using a single layer of neurons capable of learning linear separations. Decades later, Rumelhart, Hinton, and Williams [54] marked a resurgence in ANN research by introducing the backpropagation algorithm for training multi-layer networks. This innovation stimulated the deep learning revolution of the 2000s and 2010s, solidifying ANNs as a fundamental element of modern artificial intelligence [38].

The following sections are based on the books [9, 26].

2.3.1 Architecture

An ANN consists of an input layer, one or more hidden layers, and an output layer. Each neuron in a layer is connected to every neuron in the subsequent layer. See Figure 5 for a visual representation of an ANN with one hidden layer.

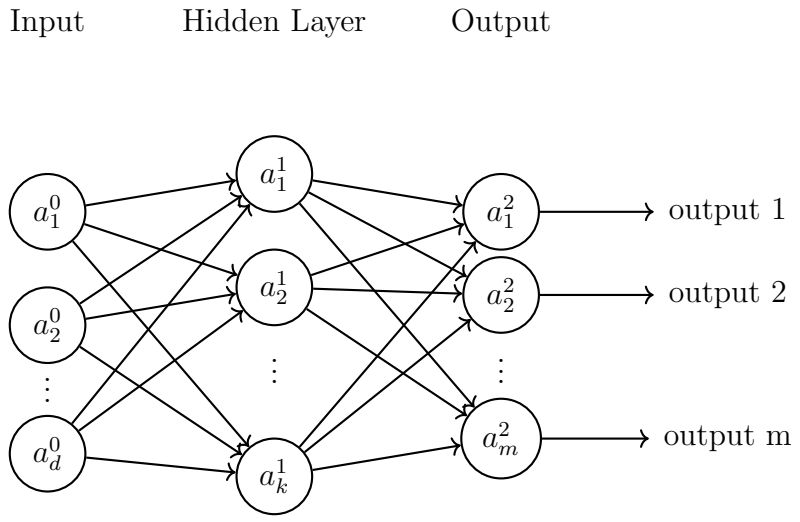

Figure 5: ANN with an input layer of d nodes, a single hidden layer with k nodes, and an output layer of m nodes. Each node is represented as a_b^c, meaning that it is the b-th node of layer number c. Own elaboration.

2.3.2 Forward Propagation

The initial stage in the operation of ANNs is forward propagation [54]. During this process, the network processes input data, transforming it progressively through interconnected layers to produce predictive outcomes. Hence, the forward propagation process computes the ANN output given an input $\mathbf{x}$.

For an ANN with L layers (including the input and output layers), the forward propagation process can be described by the following formula:

$$\mathbf{a}^{(l)} = \sigma(\mathbf{W}^{(l)}\mathbf{a}^{(l-1)} + \mathbf{b}^{(l)}) \quad \text{for} \quad l = 1, 2, \ldots, L \tag{52}$$

Here, $\mathbf{a}^{(0)} = \mathbf{x} \in \mathbb{R}^d$ is the input to the network, where d is the number of features of the input sample. $\mathbf{W}^{(l)} \in \mathbb{R}^{d_l \times d_{l-1}}$ is the weight matrix of the l-th layer, where d_l is the number of neurons in layer l. $\mathbf{b}^{(l)} \in \mathbb{R}^{d_l}$ is the bias vector for the l-th layer, and $\sigma(\cdot)$ represents the activation function applied to the linear transformation of the input.

2.3.3 Activation Function

Activation functions are a key component of ANNs since they introduce non-linearity into the model [55].

Common activation functions include the sigmoid, hyperbolic tangent (tanh), and Rectified Linear Unit (ReLU) functions. The ReLU is one of the most widely used activation functions in deep learning due to its simplicity and effectiveness. It is defined as:

$$\text{ReLU}(x) = \max(0, x) \tag{53}$$

This function outputs the input directly if it is positive; otherwise, it outputs zero.

The ReLU function has several advantages, including computational efficiency and the ability to mitigate the vanishing gradient problem [26].

2.3.4 Loss Function

ANNs use loss functions to measure the difference between the predicted output $\mathbf{a}^{(L)}$ and the actual target $\mathbf{y}$. Common choices for the loss function include the mean squared error (MSE) or the cross-entropy loss [38].

The MSE loss function is a common choice for regression problems. It calculates the average of the squares of the differences between the predicted values and the actual values across all output dimensions. The formula for MSE is:

$$\text{MSE} = \frac{1}{n}\sum_{i=1}^{n}\|\mathbf{y}_i - \hat{\mathbf{y}}_i\|_2^2 \tag{54}$$

where n is the number of samples, $\mathbf{y}_i$ is the actual target output vector, and $\hat{\mathbf{y}}_i$ is the predicted output vector.

The cross-entropy loss function is commonly used in classification problems. It measures the dissimilarity between the true distribution and the predicted distribution. For a multi-class classification problem with m output nodes, the categorical cross-entropy loss is defined as:

$$\text{Categorical Cross-Entropy} = -\frac{1}{n}\sum_{i=1}^{n}\sum_{j=1}^{m} y_{ij}\log(\hat{y}_{ij}) \tag{55}$$

where n is the number of samples, m is the number of classes (output nodes), y_{ij} is the true binary indicator (0 or 1) for class j of the i-th sample, and $\hat{y}_{ij}$ is the network's predicted probability for that same class and sample.

2.3.5 Backpropagation

Backpropagation was introduced by [54] and is used to update the weights and biases of the ANN to minimise the loss function. It involves computing the gradients of the loss function with respect to the weights and biases using the chain rule and then updating the parameters. The process of the backpropagation algorithm can be summarised as follows:

1. **Forward Pass**: The input data is propagated forward through the network, computing the output of each layer until the final output is obtained. The process is explained in Section 2.3.2.
2. **Loss Computation**: The loss function is computed based on the predicted and actual outputs, as explained in Section 2.3.4.
3. **Backward Pass**: Gradients of the loss function with respect to the weights and biases are computed using the chain rule, starting from the output layer and moving backwards through the network.
4. **Parameter Update**: The weights and biases are updated in the opposite direction of the gradients, using gradient descent or its variants [4].
5. **Iteration**: Steps 1-4 are repeated iteratively for a specified number of epochs or until convergence, gradually reducing the loss and improving the ANN's performance.

2.3.6 SVMs as Single-Layer Networks

Throughout the previous sections, we have consistently pointed out that SVMs are shallow models. This section seeks to justify why SVMs are considered shallow architectures by representing their structure as a network (see Figure 6).

The network representation of an already optimised SVM explains two interesting characteristics of the model: the shallowness of the architecture and the fact that already optimised SVMs only depend on the support vectors of the model. The shallow nature is evident as the network comprises only a single hidden layer. Furthermore, this hidden layer has m nodes, where m is the number of support vectors of the SVM. We note that this corresponds to the dual (kernel) representation, in which each hidden node is a kernel evaluation against a support vector; equivalently, the primal representation casts the same model as a single linear unit acting on the (possibly infinite-dimensional) feature map $\phi(\mathbf{x})$. The architecture proposed in this work generalises the latter, primal feature-map view.

The architecture we propose is based on expanding the shallow model to have more than one hidden layer, thus making it, ideally, more capable of learning intricate patterns in data.

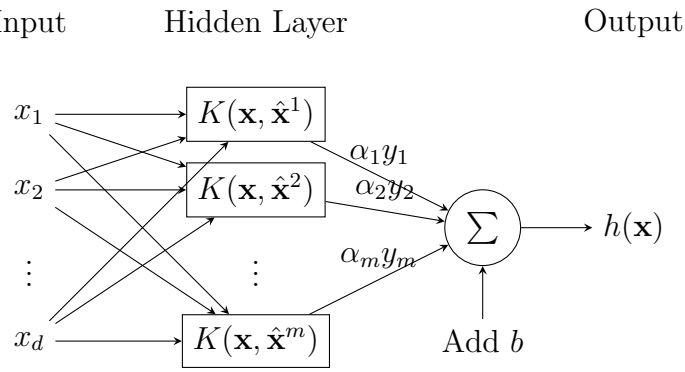

Figure 6: SVC as a single hidden layer network. The input vector is $\mathbf{x} \in \mathbb{R}^d$, where d is the number of features. $K(\mathbf{x}, \hat{\mathbf{x}}^i)$ computes the inner product between the input vector and support vector i. Support vectors are $\hat{\mathbf{x}}^i \in \mathbb{R}^d$ such that $\alpha_i > 0$, and m is the number of support vectors. Lagrange multipliers of the optimised dual SVM problem are represented as $\alpha_i \; \forall i = 1 \dots m$. The bias, b, is computed from the support vectors. $y_i \in \{-1, 1\}$ denotes the label of the support vector $\hat{\mathbf{x}}^i$. The final classification is determined by $\text{sign}(h(\mathbf{x}))$, where $h(\mathbf{x}) = \sum_{i=1}^{m} \alpha_i y_i K(\mathbf{x}, \hat{\mathbf{x}}^i) + b$. Own elaboration.

2.4 Deep Kernel Learning and the Arc-Cosine Kernel

As established in Section 2.3.6, standard SVMs can be viewed as single-layer ANNs. To bridge the gap between kernel methods and deep learning architectures, Cho and Saul [18] introduced a novel family of positive-definite kernels known as arc-cosine kernels. These kernels are specifically designed to mimic the computation of large ANNs with one or more layers of hidden units. Their positive-definiteness follows directly from their definition as an inner product of feature maps (equivalently, as the expectation in Equation 58 of a product of identical functions of $\mathbf{x}$ and $\mathbf{y}$), rather than from Bochner's theorem; indeed, arc-cosine kernels are not shift-invariant, since they depend on the angle between the inputs rather than on their difference.

The core intuition behind the arc-cosine kernel is that it computes the inner product between the outputs of an infinitely wide single-layer ANN with random Gaussian-distributed weights. To make this precise, consider a single hidden layer with weight vectors drawn independently from a standard Gaussian distribution, $\mathbf{w} \sim \mathcal{N}(0, I)$, where each hidden unit applies the non-linearity

$$g_n(z) = \Theta(z)\, z^n, \tag{56}$$

with $\Theta(\cdot)$ the Heaviside step function, defined as

$$\Theta(z) = \begin{cases} 1 & \text{if } z > 0, \\ 0 & \text{if } z \leq 0, \end{cases} \tag{57}$$

which acts as a gate that returns 1 when its argument is positive and 0 otherwise. For a given input $\mathbf{x}$, a single hidden unit therefore outputs $g_n(\mathbf{w}^T\mathbf{x}) = \Theta(\mathbf{w}^T\mathbf{x})(\mathbf{w}^T\mathbf{x})^n$: the factor $\Theta(\mathbf{w}^T\mathbf{x})$ ensures the unit activates only when the projection $\mathbf{w}^T\mathbf{x}$ is positive, in which case it outputs $(\mathbf{w}^T\mathbf{x})^n$, and otherwise outputs zero.

If such a layer has a finite number of hidden units, the inner product between the feature representations of two inputs $\mathbf{x}$ and $\mathbf{y}$ is the sum, over all units, of the products of their activations. As the number of hidden units grows to infinity, this sum, once divided by the number of units, converges to its expected value over the weight distribution; the leading factor of 2 in the definition below is the normalising constant that arises from this limit [18]. For two input vectors $\mathbf{x}, \mathbf{y} \in \mathbb{R}^d$, the n-th order arc-cosine kernel is precisely this expectation, written in integral form as:

$$k_n(\mathbf{x}, \mathbf{y}) = 2 \int \frac{e^{-\frac{\|\mathbf{w}\|^2}{2}}}{(2\pi)^{d/2}} \Theta(\mathbf{w}^T\mathbf{x})\Theta(\mathbf{w}^T\mathbf{y})(\mathbf{w}^T\mathbf{x})^n(\mathbf{w}^T\mathbf{y})^n \, d\mathbf{w} \tag{58}$$

where the term $\frac{e^{-\|\mathbf{w}\|^2/2}}{(2\pi)^{d/2}}$ is the density of the standard Gaussian distribution $\mathcal{N}(0, I)$ from which the weights are drawn, so that the integral is exactly the expectation of the product of the two units' activations over all possible weight vectors $\mathbf{w}$.

In other words, the kernel $k_n(\mathbf{x}, \mathbf{y})$ evaluates the *exact* inner product of the feature representations produced by an infinitely wide single-layer network with the activation g_n, without ever instantiating that infinite layer. The integer n is the *degree* of the kernel and selects the activation function g_n applied by the hidden units: for instance, $n = 0$ yields $g_0(z) = \Theta(z)$, a threshold activation that outputs 1 for positive inputs and 0 otherwise (the unit simply fires or stays silent), while $n = 1$ yields $g_1(z) = \Theta(z)\, z = \max(0, z)$, which is mathematically identical to the Rectified Linear Unit (ReLU) activation introduced in the previous sections. Each choice of n thus corresponds to a different infinitely wide neural layer, and Cho and Saul [18] further show that these kernels can be composed in multiple layers, mimicking the behaviour of a deep network.

While the exact computation of the arc-cosine kernel provides a powerful theoretical link between SVMs and ANNs, computing the full kernel matrix remains computationally intractable for large datasets, as discussed in Section 2.2.2.

3 State Of The Art

The integration of kernel methods and ANNs has been pursued from several distinct directions, each combining the two paradigms through a different mechanism. This chapter organises the relevant literature into five strands. We begin with the theoretical correspondence that links infinitely wide networks to kernels (Section 3.1), which provides the formal basis for the entire field and supports the arc-cosine kernel introduced in Section 2.4. We then review the two dominant constructive approaches: using an ANN as a feature extractor for a kernel machine (Section 3.2), and stacking kernel machines into explicitly deep architectures (Section 3.3), the latter being the line of work that this thesis extends. We subsequently discuss deep architectures built without backpropagation (Section 3.4), which motivate the training philosophy adopted here, and recent kernel machines that learn features directly (Section 3.5). Finally, Section 3.6 positions the present work with respect to this landscape.

3.1 The Neural-Kernel Correspondence

The deepest connection between the two paradigms is theoretical: under certain limits, ANNs and kernel methods are formally equivalent. This observation is the foundation on which the rest of the field is built.

The starting point of this strand is the work of Cho and Saul [17], who introduced the arc-cosine kernels already described in Section 2.4. By showing that the inner product of the features computed by an infinitely wide single-layer network with random Gaussian weights admits a closed-form kernel expression, they provided an explicit bridge between a neural layer and a kernel, and further showed that such kernels can be composed in multiple layers to emulate a deep network.

This was later generalised in two influential ways. The first is the *Neural Network Gaussian Process* (NNGP) view: as the width of a randomly initialised network tends to infinity, the function it represents converges to a Gaussian process whose covariance is the NNGP kernel, so that Bayesian inference with the network reduces to kernel regression [49, 40]. The second is the *Neural Tangent Kernel* (NTK) of Jacot et al. [34], which characterises not the network at initialisation but its *training dynamics*: in the infinite-width limit, gradient-descent training of a network behaves like kernel gradient descent with a fixed kernel, the NTK. Together, these results establish that under certain conditions, wide ANNs and kernel methods are two views of the same object. While these analyses are primarily theoretical and are not in themselves practical learning algorithms, they justify the central premise of this work: that a kernel built to imitate a neural layer is a principled building block.

3.2 Deep Kernel Learning: Networks as Feature Extractors

A first constructive way of combining the two paradigms is to place a deep ANN in front of a kernel machine, using the network to learn a rich, data-dependent representation and the kernel to perform the final prediction. The most prominent example is the *Deep Kernel Learning* (DKL) framework of Wilson et al. [65], in which the inputs of a base kernel are transformed by a deep architecture, and the parameters of the resulting deep kernel are learned jointly through the marginal likelihood of a Gaussian process. This yields a model that retains the non-parametric flexibility and uncertainty estimates of Gaussian processes while benefiting from the representational power of deep networks, and that scales to large datasets through inducing-point and structure-exploiting approximations.

This idea has been extended in several directions, including kernels with recurrent structure for sequential data [56] and latent-variable formulations that regularise the learned representation [42]. More recently, the *Deep Kernel Machines* (DKM) of Yang et al. and Milsom et al. [66, 46] revisit the infinite-width limit itself, modifying it so that intermediate-layer representations are no longer fixed but adapt to the data, and derive a practical deep learning algorithm from deep Gaussian processes, including a convolutional variant competitive with ANNs on image benchmarks. A common characteristic of this entire family is that the model is trained *end-to-end* by gradient-based optimisation of a single global objective, typically a (variational) marginal likelihood. This contrasts with the convex, layer-wise training pursued in this thesis.

3.3 Deep Architectures of Stacked Kernel Machines

The strand most directly related to this thesis builds depth not by feeding a network into a single kernel, but by *stacking kernel machines into multiple layers*, so that each layer transforms the representation produced by the previous one. This is the natural kernel-based analogue of a deep network, and it is the family that the present work belongs to and extends.

An early and influential idea was the analytic composition of arc-cosine kernels [17]: because each arc-cosine kernel corresponds to a neural layer, composing L of them yields a kernel that mimics an L-layer network. This gives rise to *multilayer kernel machines*, in which the kernel itself is deep. While elegant, this analytic composition does not scale to large datasets, since it still requires the full kernel matrix; subsequent work explored core vector machines as a scaling-up mechanism for deep arc-cosine support vector machines [3], addressing precisely the scalability problem that motivates the present thesis, though through a different route.

In parallel, several authors built deep models from SVMs as the base unit. Wiering et al. proposed the Multi-Layer Support Vector Machine (MLSVM) and the Neural Support Vector Machine [64, 63], deep SVM models trained by a gradient-

based rule on a min-max formulation of the optimisation problem, reporting improvements over a standard SVM on regression benchmarks. More recently, Wang et al. introduced the *SVM-based Deep Stacking Network* (SVM-DSN) [61], which organises linear SVMs in the Deep Stacking Network architecture of Deng and colleagues [22, 21] and introduces a backpropagation-like layered tuning scheme to optimise all base-SVMs jointly while preserving their individual convexity. Related constructions include the D-SVM of Abdullah et al. [1], which feeds the kernel activations of support vectors as inputs to the next layer and combines several such models in an ensemble, and the stacked-SVM feature extractor of Kim et al. [36], in which higher-layer SVMs were observed to generalise better than lower ones.

Of particular relevance to this thesis is the line of work by Mehrkanoon and Suykens on *deep hybrid neural-kernel networks* [45]. Their model uses an explicit feature map based on RFFs to make the transition between the neural and kernel components straightforward, and crucially to make the model scalable by solving the optimisation problem in the primal rather than forming the kernel matrix. Stacking several such hybrid layers yields a deep architecture, and follow-up work extended this with additional block types such as average, maxout, and convolutional kernel blocks. The use of an explicit random-feature map to obtain scalability and primal-form training is mechanically the closest precedent to the approach taken in this work; the present thesis differs in its use of the arc-cosine feature map and in a fully greedy, layer-wise training procedure.

Building directly upon these scalable, primal-form approximations, recent work by Acero and Belanche [2] proposed a greedy Multi-Layer Support Vector Machine (ML-SVM) in which linear SVMs are stacked across layers, each layer transforming its input through a Gaussian RBF kernel approximation based on RFFs and feeding the resulting SVM outputs to the next layer. The architecture is trained greedily, one layer at a time, solving only linear optimisation problems and avoiding both kernel matrices and backpropagation, and was shown to match a standard RBF SVM on large datasets while training substantially faster. A structural limitation of this design, however, is that only the low-dimensional SVM decision values are propagated between layers, a single value per class, so that the rich, high-dimensional representation built inside each layer is discarded before reaching the next. This thesis identifies this information bottleneck as a central obstacle to making depth consistently beneficial, and the architecture proposed here is explicitly designed to address it, using an arc-cosine rather than an RBF feature map.

3.4 Deep Learning Without Backpropagation

A separate but conceptually important strand questions whether depth requires backpropagation at all. The most prominent example is the *Deep Forest* (gcForest) of Zhou and Feng [68], which constructs a deep model from non-differentiable modules, namely cascaded layers of random forests. The authors conjecture that the

effectiveness of deep models stems from three characteristics, layer-by-layer processing, in-model feature transformation, and sufficient model complexity, none of which intrinsically requires differentiable modules or gradient-based end-to-end training. Each layer is trained independently and its outputs are concatenated with the original features and passed to the next layer, so that the architecture grows greedily without ever backpropagating an error signal. Related ideas include Forward Thinking, which builds deep networks one layer at a time by passing the outputs of each layer forward [31], and the original Deep Stacking Networks [22], whose blocks are trained layer by layer and which provided the architectural template later adopted by SVM-DSN.

This strand is relevant to the present work not because it uses kernels, but because it validates the design philosophy adopted here: a deep architecture can be built by stacking easy-to-train, individually optimal modules in a greedy, layer-wise fashion, retaining the layer-by-layer feature transformation that gives deep models their power while avoiding the non-convex optimisation and tuning difficulties of backpropagation.

3.5 Feature-Learning Kernel Machines

A final and more recent strand seeks to endow kernel machines with the feature-learning ability that was long considered the exclusive advantage of ANNs. Radhakrishnan et al. [50] introduced *Recursive Feature Machines* (RFMs), kernel machines that explicitly learn features by alternating between two steps: fitting a kernel predictor, and reweighting the input features using the *Average Gradient Outer Product* (AGOP) of that predictor, a statistical estimator that up-weights the directions most relevant to the output. Iterating this process allows a kernel machine to recursively refine its representation, and the authors posit, through the Deep Neural Feature Ansatz, that this same mechanism underlies feature learning in deep ANNs. Subsequent analysis by Gan and Poggio [24] showed that AGOP can be understood as a greedy approximation to gradient descent, which is of direct relevance to the greedy training strategy adopted in this thesis. A related convolutional construction is given by the Convolutional Kernel Networks of Mairal et al. [43], which approximate a hierarchical kernel through a network-like architecture of feature maps.

3.6 Positioning of This Work

The methods reviewed above show that the gap between kernel methods and ANNs has been bridged from several angles, yet no existing approach occupies the specific intersection targeted by this thesis. The Gaussian-process deep kernels of Section 3.2 and the recent deep kernel machines are trained end-to-end by gradient methods, sacrificing the convexity that makes SVMs attractive. Among the stacked-SVM models of Section 3.3, the multilayer arc-cosine kernel relies on analytic kernel composi-

tion that does not scale, while SVM-DSN and the Neural Support Vector Machine reintroduce a backpropagation-like global tuning step that forfeits the simplicity of purely greedy training. The backpropagation-free architectures of Section 3.4 are not kernel-based, and the feature-learning kernels of Section 3.5 operate on a single shallow kernel rather than a deep stack.

The closest precedent is the greedy ML-SVM of Acero and Belanche [2], which is already convex, greedy, and scalable. It differs from the present work in two essential respects. First, it propagates only the low-dimensional SVM decision values between layers, creating the information bottleneck discussed in Section 3.3. Second, it relies on a Gaussian RBF kernel approximation.

This thesis aims to fill that gap by developing a deep architecture that is simultaneously built upon the convex, margin-based formulation of support vector machines; scalable to large datasets; and trained in a fully greedy, layer-wise manner that avoids backpropagation entirely. The remainder of this document details this architecture and evaluates each of its design decisions empirically.

4 The Deep Convex Kernel Machine

This chapter introduces the proposed architecture, the Deep Convex Kernel Machine (DCKM), a deep, convex, and backpropagation-free model that stacks support vector machines into multiple layers. Section 4.1 describes the architecture and the principle that distinguishes it from its predecessor. Section 4.2 specifies the arc-cosine feature map that gives each layer its interpretation as an infinite-width ANN layer. Section 4.3 presents the greedy layer-wise learning algorithm, Section 4.4 the prediction procedure, and Section 4.5 the analysis of computational complexity. Finally, Section 4.6 shows that the architecture is in fact kernel-agnostic, the arc-cosine map being one principled choice among others.

4.1 Architecture

4.1.1 Motivation: Avoiding Representation Collapse

The point of departure for this work is the greedy multilayer SVM (ML-SVM) of Acero and Belanche [2], described in Section 3.3. As established there, that architecture propagates only the scalar SVM decision value between consecutive layers. Since the decision value lies in R for regression, and in R^K for a K-class problem, the high-dimensional representation built inside each layer through the random-feature map is discarded before reaching the next layer. We refer to this loss of information as *representation collapse*: the inter-layer signal is constrained to a dimension fixed by the task rather than by the model's capacity, so that adding layers contributes little, as the experiments of Acero and Belanche on large datasets confirm.

The architecture proposed here, DCKM, resolves this limitation by a single structural change: instead of forwarding the scalar decision values, it forwards a *learned linear projection of the full feature representation*. The dimension of the inter-layer representation then becomes a free parameter of the model, independent of the number of classes, and representation collapse becomes structurally impossible.

4.1.2 Layer Structure

Let $\mathbf{X}^{(0)} = \mathbf{X} \in R^{n \times d}$ denote the input data, with n samples and d features. The architecture consists of L stacked blocks. Each block ℓ applies two successive transformations to its input $\mathbf{X}^{(\ell)}$:

1. A *non-linear* transformation through a random-feature map, $\mathbf{X}^{(\ell)} \mapsto \mathbf{\Phi}^{(\ell)} \in R^{n \times P}$, where P is the number of random features. The map is parameterised by a weight matrix $\mathbf{\Omega}^{(\ell)}$ sampled according to the chosen kernel (Section 4.2).

2. A *linear* transformation, $\mathbf{\Phi}^{(\ell)} \mapsto \mathbf{\Phi}^{(\ell)}\mathbf{W}^{(\ell)} = \mathbf{X}^{(\ell+1)}$, learned by a block of m parallel linear SVMs.

Each of the m SVMs in block ℓ is trained on the same feature map $\mathbf{\Phi}^{(\ell)}$ and produces a weight vector $\mathbf{w}_k^{(\ell)} \in R^P$. These are collected column-wise into the projection matrix

$$\mathbf{W}^{(\ell)} = \left[\mathbf{w}_1^{(\ell)}, \ldots, \mathbf{w}_m^{(\ell)}\right] \in R^{P\times m}, \tag{59}$$

so that the inter-layer representation $\mathbf{X}^{(\ell+1)} = \mathbf{\Phi}^{(\ell)}\mathbf{W}^{(\ell)} \in R^{n\times m}$ has dimension m. The design intent is $P \gg m$: the block expands the data into a high-dimensional feature space and then compresses it into a learned m-dimensional subspace, retaining far more information than a single scalar decision value. After the final block, a single linear SVM is trained on $\mathbf{X}^{(L)}$ to produce the model's output.

This structure mirrors a neural-network layer, a non-linear map followed by a learned linear projection, with one essential difference: each linear map $\mathbf{W}^{(\ell)}$ is the solution of m convex optimisation problems, rather than the result of gradient descent. There is no backpropagation, and the margin-based guarantees of the SVM are preserved at every layer.

The use of m parallel SVMs per block, rather than one, provides the projection matrix $\mathbf{W}^{(\ell)}$ with m columns and thus controls the dimension of the inter-layer representation. How these m SVMs are made to differ, for instance by assigning them distinct regularisation constants, is a design choice that influences the diversity of the learned projection; its effect is studied empirically in Chapter 5.5.2 rather than assumed here.

4.2 The Arc-Cosine Feature Map

The defining choice of the proposed architecture is the feature map used in each block. We employ the arc-cosine kernel of Cho and Saul [18], introduced in Section 2.4, because it endows each block with a precise interpretation as an infinite-width ANN layer. Recall that the n-th order arc-cosine kernel is defined directly as an expectation over Gaussian weights $\mathbf{w} \sim \mathcal{N}(0, I)$:

$$k_n(\mathbf{x}, \mathbf{z}) = E_{\mathbf{w}\sim\mathcal{N}(0,I)}\left[\phi_{\mathbf{w}}^{(n)}(\mathbf{x})\,\phi_{\mathbf{w}}^{(n)}(\mathbf{z})\right], \qquad \phi_{\mathbf{w}}^{(n)}(\mathbf{x}) = \sqrt{2}\,\Theta(\mathbf{w}^\top\mathbf{x})(\mathbf{w}^\top\mathbf{x})^n. \tag{60}$$

4.2.1 The Random Feature Map

Because the kernel is already written as an expectation, it admits an immediate Monte Carlo approximation, as described in general terms in Section 2.2.2. Drawing

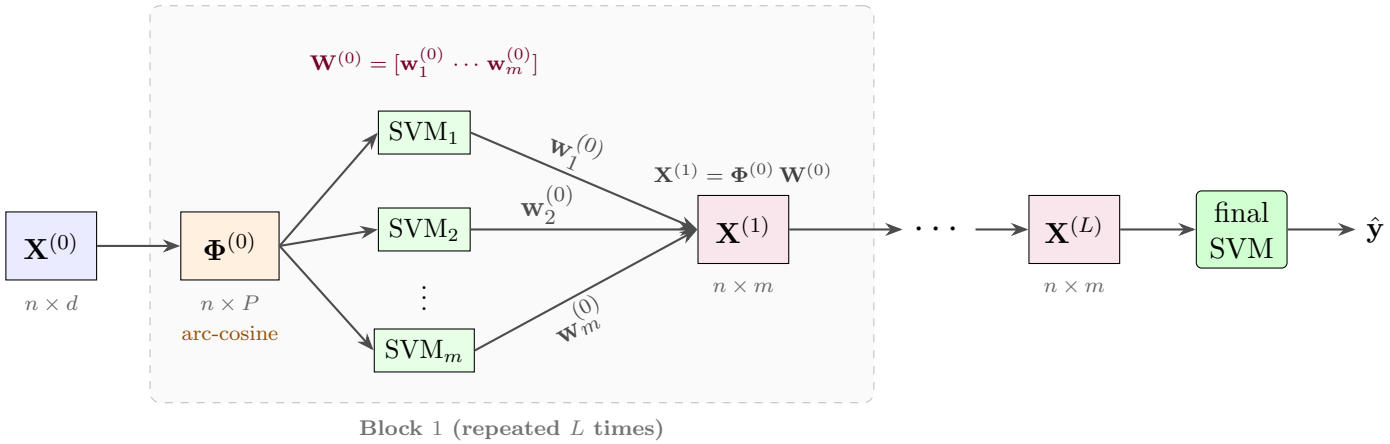

Figure 7: The DCKM architecture. The input $\mathbf{X}^{(0)} \in R^{n\times d}$ passes through L stacked blocks. Within block ℓ, an arc-cosine random-feature map expands the input into $\mathbf{\Phi}^{(\ell)} \in R^{n\times P}$; a block of m parallel linear SVMs is trained on $\mathbf{\Phi}^{(\ell)}$, and their weight vectors form the columns of the projection matrix $\mathbf{W}^{(\ell)} \in R^{P\times m}$, which compresses the representation to $\mathbf{X}^{(\ell+1)} = \mathbf{\Phi}^{(\ell)}\mathbf{W}^{(\ell)} \in R^{n\times m}$. Because the inter-layer dimension m is a free parameter rather than the number of classes, representation collapse is avoided. A final linear SVM maps $\mathbf{X}^{(L)}$ to the prediction $\hat{\mathbf{y}}$. Every linear map is the solution of a convex problem; there is no backpropagation.

$\boldsymbol{\omega}_j \sim \mathcal{N}(0, I)$ independently for $j = 1, \ldots, P$, the feature map is

$$\mathbf{\Phi}_j^{(\ell)}(\mathbf{x}) = \sqrt{\frac{2}{P}}\,\max\bigl(0,\, \boldsymbol{\omega}_j^\top \mathbf{x}^{(\ell)}\bigr)^n . \tag{61}$$

The inner product $\mathbf{\Phi}^{(\ell)}(\mathbf{x})^\top \mathbf{\Phi}^{(\ell)}(\mathbf{z})$ is then an unbiased estimator of $k_n(\mathbf{x}, \mathbf{z})$, convergent as $P \to \infty$ by the law of large numbers. This construction is a direct application of Monte Carlo integration to an expectation that Cho and Saul already identify; no phase variable is required, since the features are real and non-negative by construction.

4.2.2 Why Bochner's Theorem Is Not Needed

The RFFs commonly used to approximate the RBF kernel are justified by Bochner's theorem, which applies only to shift-invariant kernels. Arc-cosine kernels are not shift-invariant: $k_n(\mathbf{x}, \mathbf{z})$ depends on the norms of $\mathbf{x}$ and $\mathbf{z}$ and the angle between them, not on their difference, so Bochner's theorem does not apply. This raises three questions, each resolved without it. *Positive-definiteness* follows directly from the integral representation: any function expressible as $E[\phi_{\mathbf{w}}(\mathbf{x})\phi_{\mathbf{w}}(\mathbf{z})]$ is a valid covariance and hence positive-definite. *Unbiasedness* of the estimator follows from

the same expectation form by the law of large numbers. The *sampling distribution* is not a free choice to be fixed by a theorem, as it is for shift-invariant kernels, but is prescribed directly as $\mathcal{N}(0, I)$ by the integral representation. Consequently, the arc-cosine map requires no bandwidth parameter: the scale of the kernel is determined entirely by the data norms, and since the data is standardised before training, $\mathcal{N}(0, I)$ is both the theoretically prescribed and the practically sufficient choice. This is a simplification relative to standard RFFs.

4.2.3 Theoretical Interpretation and Its Scope

Using arc-cosine features of degree $n = 1$, for which the activation is $\max(0, u)$, the ReLU, gives each block a precise meaning: the non-linear map $\mathbf{\Phi}^{(\ell)}$ is a finite-sample approximation to the arc-cosine kernel of degree 1, which is exactly the kernel induced by an infinite-width single-layer ReLU network with $\mathcal{N}(0, I)$ weights. The projection matrix $\mathbf{W}^{(\ell)}$ is the learned component, playing the role of the output weights of that infinite network, determined by a convex SVM solve rather than by backpropagation. This interpretation is the reason the arc-cosine map is adopted as the definitive form of the architecture: it realises, in a convex and backpropagation-free model, a finite-sample analogue of a deep ReLU network.

The scope of this claim must be stated precisely. The equivalence holds *per block*: each block individually approximates one arc-cosine kernel layer. It does not extend to the full stacked architecture. The composed kernel of Cho and Saul is defined by a recursion that uses exact kernel values at each layer, whereas DCKM forwards a learned linear projection $\mathbf{X}^{(\ell+1)} = \mathbf{\Phi}^{(\ell)}\mathbf{W}^{(\ell)}$ whose values are determined by the training labels, not by the kernel recursion. The stacked architecture therefore does not approximate the L-fold composed arc-cosine kernel, and no such claim is made.

4.2.4 Practical Considerations

Two practical points follow from this choice. First, the architecture requires no bandwidth parameter, as argued above. Second, the features $\max(0, \boldsymbol{\omega}^\top \mathbf{x})^n$ are unbounded and grow with $\|\mathbf{x}\|$; consequently, the inter-layer representation $\mathbf{X}^{(\ell+1)}$ must be standardised before the next feature map to prevent scale drift across layers, using statistics fixed at training time. Standard scaling between blocks is sufficient.

4.3 Learning Algorithm

The architecture is trained greedily, one block at a time. Each block is trained to completion before the next is constructed, using the output of the previous block as its input. Because each block reduces to the training of independent linear SVMs, each of which is a convex problem, the procedure inherits the optimisation guarantees

of the SVM and requires no global, end-to-end optimisation. Algorithm 1 summarises the procedure.

Algorithm 1 DCKM training

Require: Training data $(\mathbf{X}, \mathbf{y})$; number of layers L; SVMs per block m; regularisations $C_1, \ldots, C_m$; random features P

Ensure: Per-block models $\{\mathbf{W}^{(\ell)}, \mathbf{\Omega}^{(\ell)}\}_{\ell=0}^{L-1}$, final SVM $\mathbf{w}^{(L)}$

$\mathbf{X}^{(0)} \leftarrow \mathbf{X}$
for $\ell = 0, \ldots, L-1$ **do**
  Sample $\mathbf{\Omega}^{(\ell)} \sim \mathcal{N}(0, I)$ ▷ arc-cosine weights
  $\mathbf{\Phi}^{(\ell)} \leftarrow \phi(\mathbf{\Omega}^{(\ell)}, \mathbf{X}^{(\ell)}) \in R^{n \times P}$ ▷ feature map, Eq. 61
  for $k = 1, \ldots, m$ **do**
    $\mathbf{w}_k^{(\ell)} \leftarrow \textsc{LinearSVM}(\mathbf{\Phi}^{(\ell)}, \mathbf{y}; C_k)$
  end for
  $\mathbf{W}^{(\ell)} \leftarrow [\mathbf{w}_1^{(\ell)}, \ldots, \mathbf{w}_m^{(\ell)}]$
  $\mathbf{X}^{(\ell+1)} \leftarrow \mathbf{\Phi}^{(\ell)} \mathbf{W}^{(\ell)}$ ▷ forward the projection, not the decision values
end for
$\mathbf{w}^{(L)} \leftarrow \textsc{LinearSVM}(\mathbf{X}^{(L)}, \mathbf{y}; C)$

The weight matrices $\mathbf{\Omega}^{(\ell)}$ sampled for the feature maps must be stored, since they are required to reproduce the same transformations at prediction time. Each linear SVM is solved in its primal form (Section 4.5), which is the source of the architecture's scalability.

4.4 Prediction

Prediction applies the stored transformations of each block in sequence. Given a new input $\mathbf{x}$, the representation is propagated forward through the L blocks: at each block ℓ, the stored sampling matrix $\mathbf{\Omega}^{(\ell)}$ defines the feature map $\mathbf{\Phi}^{(\ell)}$, and the stored projection $\mathbf{W}^{(\ell)}$ produces the next representation $\mathbf{x}^{(\ell+1)} = \mathbf{\Phi}^{(\ell)}(\mathbf{x})\, \mathbf{W}^{(\ell)}$, which is standardised using the training-time statistics before the next block. The final linear SVM is then applied to $\mathbf{x}^{(L)}$ to produce the prediction.

4.5 Computational Complexity

The scalability of DCKM rests on a single principle: the Gram matrix is never formed. We develop the argument here, as it is central to the architecture's applicability in the large-n regime.

An exact kernel SVM requires the $n \times n$ kernel matrix, at a cost of $O(n^2 d)$ to build and between $O(n^2)$ and $O(n^3)$ to solve (Section 2.2.2); both costs grow at least quadratically in the number of samples n, which is precisely what renders exact kernel SVMs intractable for large datasets. DCKM replaces this with an explicit

random-feature map. By construction, the random-feature matrix $\mathbf{\Phi}^{(\ell)} \in R^{n \times P}$ satisfies $\mathbf{\Phi}^{(\ell)}\mathbf{\Phi}^{(\ell)\top} \approx \mathbf{K}$, so a linear SVM trained on the rows of $\mathbf{\Phi}^{(\ell)}$ approximates a kernel SVM without ever materialising $\mathbf{K}$.

Concretely, each linear SVM is solved in its primal form,

$$\min_{\mathbf{w} \in R^P} \frac{1}{2}\|\mathbf{w}\|^2 + C \sum_{i=1}^{n} L\Big(y_i,\, \mathbf{w}^\top \mathbf{\Phi}^{(\ell)}_{i\cdot}\Big)\,, \tag{62}$$

which operates directly on the rows of $\mathbf{\Phi}^{(\ell)}$ and never forms the product $\mathbf{\Phi}^{(\ell)}\mathbf{\Phi}^{(\ell)\top}$. The primal and dual solutions are equivalent by strong duality, but only the dual requires the Gram matrix; solving in the primal is what avoids the quadratic cost.

The choice of the primal is deliberate and tied to the regime the architecture targets. The primal variable $\mathbf{w}$ lies in R^P, so the cost of the primal solve scales with the number of random features P; the dual variable $\boldsymbol{\alpha}$ lies in R^n and depends on the data through the $n \times n$ Gram matrix $\mathbf{\Phi}^{(\ell)}\mathbf{\Phi}^{(\ell)\top}$. The primal is therefore the more efficient formulation precisely when $n \gg P$, and the dual when $P \gg n$. Since DCKM targets the large-sample regime, in which n may reach the order of millions while P is a fixed hyperparameter of moderate size, the condition $n \gg P$ holds by design, and the primal is the appropriate choice. Crucially, solving the dual would reintroduce the $n \times n$ Gram matrix that the random-feature approximation was introduced to avoid, reinstating the quadratic memory and super-quadratic time cost of the exact kernel SVM and thereby negating the entire computational benefit of the approach. The random-feature map and the primal solve are thus two halves of a single design: once the kernel is made explicit through the feature map $\mathbf{\Phi}^{(\ell)}$, the kernel trick, whose natural home is the dual, is no longer required, and the problem is most naturally and most cheaply solved as a linear model trained directly on the explicit features. In the opposite regime $P > n$ the dual would be preferable, but that regime lies outside the scope of this work, whose aim is to remain tractable as n grows large.

The cost of a single block is therefore the sum of three linear-in-n terms: constructing $\mathbf{\Phi}^{(\ell)}$ costs $O(nPd_\ell)$, where d_ℓ is the input dimension of the block ($d_0 = d$, and $d_\ell = m$ for $\ell \geq 1$); training the m linear SVMs costs, for each, time linear in n for a fixed P, since the solver operates by repeated passes over the n rows of $\mathbf{\Phi}^{(\ell)}$ at a per-pass cost of $O(nP)$; and forming the projection $\mathbf{\Phi}^{(\ell)}\mathbf{W}^{(\ell)}$ costs $O(nPm)$. Summing over L blocks, the total training cost is linear in n for fixed P, m, and L. This is the decisive contrast with the exact kernel SVM: DCKM trades the quadratic-to-cubic dependence on n for a linear one, at the price of approximating the kernel through P random features, which makes the architecture tractable in the large-n regime where exact SVMs cannot be applied.

4.6 Generality of the Architecture

The arc-cosine map of Section 4.2 is the definitive instantiation of the proposal, but it is not intrinsic to it. The architecture, a stack of blocks, each pairing a non-linear feature map with a learned linear SVM projection, is in fact *kernel-agnostic*: any feature map may be substituted, provided the dimensional structure of the projection is respected. We make this generality precise, since it clarifies which part of the proposal is the contribution (the architecture) and which is a principled choice within it (the arc-cosine kernel).

The general requirement is simply that each block produce an inter-layer representation $\mathbf{X}^{(\ell+1)} \in R^{n \times m}$, in which each of the m SVMs of the block contributes one column. Two cases arise. If all m SVMs in a block share a single feature map $\mathbf{\Phi}^{(\ell)} \in R^{n \times P}$, their weight vectors are conformable, and the block reduces to the compact matrix form $\mathbf{X}^{(\ell+1)} = \mathbf{\Phi}^{(\ell)}\mathbf{W}^{(\ell)}$ of Equation 59; this is the case used throughout this thesis. If instead each SVM k uses its own feature map $\mathbf{\Phi}_k^{(\ell)} \in R^{n \times P_k}$, the weight vectors have differing dimensions and cannot be stacked into a single matrix; the block is then defined by the per-SVM projections $\mathbf{X}_{\cdot k}^{(\ell+1)} = \mathbf{\Phi}_k^{(\ell)}\mathbf{w}_k^{(\ell)}$, concatenated column-wise. In either case the inter-layer representation has the same shape $n \times m$, and the next block proceeds identically.

This freedom permits several variants. The most natural alternative is the *RBF variant*, in which each block uses the random Fourier feature map of the Gaussian kernel [51], with a per-layer bandwidth estimated by the median heuristic; its cosine features are bounded in $[-1, 1]$, which removes the need for inter-layer standardisation. This variant lacks the infinite-width ReLU interpretation of the arc-cosine map, but is architecturally identical, and serves in Chapter 5.5.2 as a control that isolates the effect of the kernel choice from that of the architecture. More generally, different kernels may be used at different layers, or even at different SVMs within a layer, allowing the representation to be refined by heterogeneous transformations. We do not pursue these heterogeneous configurations empirically; we note them to make clear that the proposed contribution is the stacking architecture and its projection mechanism, of which the arc-cosine kernel is the theoretically motivated, but not the only possible, realisation.

5 Experiments

This chapter evaluates the proposed architecture empirically. We restrict the study to classification. The choice is deliberate: classification benchmarks are more numerous and better standardised than regression ones, and the SVM, on which every block of the architecture rests, was formulated for classification in the first place [19]. Working in this setting keeps the architecture aligned with the theory of its components and gives access to a broad and well-characterised pool of datasets.

The four experiments build on one another. Experiment 1 establishes that the base block, before any width or depth is added, has the non-linear capacity the later experiments assume. Experiment 2 fixes how diversity should be injected among the parallel SVMs within a block. Experiment 3 isolates the two structural parameters, width m and depth L, and asks where each adds value. Experiment 4 places the resulting architecture against standard and deep-kernel baselines.

Metrics. We report classification accuracy and the macro-averaged F_1 score. Accuracy is the fraction of correctly classified test instances. The macro-F_1 score is the unweighted mean of the per-class F_1 scores, each the harmonic mean of precision and recall for that class; averaging without class weights makes the metric sensitive to performance on minority classes, which accuracy alone can mask. We therefore use macro-F_1 as the headline metric on imbalanced datasets and accuracy on balanced ones, reporting both throughout.

Statistical testing. Where several methods are compared across a heterogeneous panel of datasets, we use the Friedman test to assess whether their rankings differ beyond chance, followed by the Nemenyi post-hoc procedure to identify which pairs are separated [20]. The Friedman test ranks the methods on each dataset and tests the null hypothesis that the average ranks are equal; the Nemenyi procedure declares two methods distinguishable when their average ranks differ by more than the critical difference at a given significance level. This pairing is the standard non-parametric protocol for comparing classifiers over multiple datasets, as it makes no distributional assumptions about the metric.

Protocol. Unless stated otherwise, the total random-feature budget per block is fixed at $P = 1000$, the input features are standardised, and each configuration is run over three random seeds, with results reported as mean $\pm$ standard deviation. Experiment-specific grids and deviations from this protocol are stated in the corresponding section.

5.1 Datasets

The experiments draw on three families of datasets: synthetic generators with known structure, small standard benchmarks, and large real-world tabular problems. We summarise here the datasets used across the experiments; Table 1 gives their size, dimension, and number of classes, together with their source. Experiment-specific subsets are described in place.

The synthetic generators give controlled problems whose decision boundary is known by construction. The *spiral* dataset separates two interleaving spiral arms, a boundary that winds through the plane and cannot be captured by a single linear or shallow non-linear rule. The *checkerboard* dataset assigns labels by a parity rule over a two-dimensional grid, producing a high-frequency boundary that alternates across cells. The *radial* dataset labels points by a radial threshold combined with a feature interaction, giving a curved multi-region boundary. In every case the informative structure lives in a low-dimensional subspace, optionally embedded in additional noise dimensions, which lets us separate genuine non-linear capacity from dimensionality effects. All three are generated locally.

The real datasets are obtained through the OpenML platform [8] and originate from a mix of sources. Several are classic problems from the University of California, Irvine (UCI) repository [5], including *Poker Hand* [15], *Covertype* [10], *Chess (KRK)* [6], the physics datasets *HIGGS* and *MiniBooNE* [62, 52], *Letter* [58], and the standard small benchmarks (*breast-cancer*, *ionosphere*, *ecoli*, *glass*, *spambase*, *magic*, *adult*, *bank-marketing* [69, 57, 48, 25, 33, 12, 7, 47]). Two come from machine-learning challenges rather than UCI: *madelon*, an artificial problem with five informative features hidden among noise, was introduced for the NIPS 2003 feature-selection challenge [29], and *jannis* is a multiclass tabular problem from the AutoML challenge series [30]. Finally, *MNIST* is the standard handwritten-digit image benchmark in flattened-pixel form [39].

These datasets are chosen to span a wide range of regimes. The real tabular problems used to probe depth and width in Experiment 3 share a defining property: their labels are near-deterministic functions of the inputs with a highly non-smooth decision boundary, the regime the architecture is designed for. The remaining datasets, used in the benchmark of Experiment 4, broaden the comparison across sample size, dimension, and class count.

5.2 Experiment 1: Overfitting Capacity

5.2.1 Methodology

The mechanisms studied later in this chapter (the use of multiple SVMs per block and the use of multiple of blocks) are designed to enlarge the model capacity to fit complex functions, or by regularising a model that is already flexible enough.

Dataset	Source	n	d	Classes
Synthetic				
Radial	generated	1,500	50	2
Spiral	generated	1,500	50	2
Checkerboard	generated	1,500	50	2
Spirals	generated	120,000	2	2
Real, small benchmarks				
Breast cancer	UCI	286	9	2
Glass	UCI	214	9	6
Ecoli	UCI	336	7	8
Ionosphere	UCI	351	34	2
Madelon	NIPS 2003	2,600	500	2
Spambase	UCI	4,601	57	2
Real, large tabular and image				
Magic	UCI	19,020	11	2
Letter	UCI	20,000	16	26
Bank-marketing	UCI	45,211	51	2
Adult	UCI	48,842	14	2
MNIST	LeCun	70,000	784	10
Jannis	AutoML	83,733	54	4
HIGGS	UCI	98,050	28	2
MiniBooNE	UCI	130,064	50	2
Covertype	UCI	581,012	54	7
Real, used in Experiment 3 screen				
Chess (KRK)	UCI	28,056	6	18
Poker Hand	UCI	1,025,010	10	10

Table 1: Datasets used across the experiments, with number of instances n, input dimension d, and number of classes. Synthetic datasets are generated locally; the real datasets are retrieved through OpenML and originate from the UCI repository, the NIPS 2003 feature-selection challenge (madelon), the AutoML challenge series (jannis), or LeCun's MNIST benchmark. The dimension d here is the encoded feature count (after one-hot encoding of categorical variables), whereas Table 1 reports the raw count; this accounts for CoverType ($54 \to 98$) and Chess (KRK) ($6 \to 23$).

Experiment 1 establishes the prerequisite on which a later analysis depends: that the base architecture, before any width or depth is added, is already capable of fitting data properly. By first showing that the base architecture can drive its training error to zero on problems that other simple models cannot fit, we fix the baseline against which the effects of width and depth in the following experiments can be interpreted.

We show that this capacity originates in the random-feature map rather than in the linear SVM, and that it is governed by the feature dimension P, the only capacity parameter varied in this experiment (m and L are fixed at their minimal

values, $m = 1$, $L = 1$).

5.2.2 Experimental Design

We compare the training and test accuracy of baseline reference models against the proposed model with a single SVM per block ($m = 1$, $L = 1$), as the number of random features P is increased. The capacity should be a property of the feature-map mechanism rather than of one particular kernel, so we run the proposed model with both the arc-cosine and the Radial Basis Function (RBF) kernel. As baseline references we use a linear SVM and a depth-three decision tree: the first cannot represent non-linear boundaries at all, the second only coarse axis-aligned ones, so neither should be able to fit the training set.

To make *fitting the training set* a meaningful test of non-linear capacity, the data must be hard for structural reasons rather than because of dimensionality, and the result must not depend on a single synthetic generator. We therefore use three structurally different problems, each with $n = 1500$ training samples and $d = 50$ features of which only a few are informative, the remainder being irrelevant Gaussian noise (see Figure 8). The first, radial, labels a point by a radial threshold combined with a two-way product, giving a curved, multi-region boundary. The second, spiral, separates two interleaving spirals, a boundary that winds through the plane. The third, checker, is a checkerboard parity over a two-dimensional informative subspace, a high-frequency boundary. In every case we keep $d < n$, so the data is not trivially separable by a hyperplane in the input space; this rules out the possibility that a model fits the training set for reasons unrelated to the problem's structure. A model that drives training error to zero on these problems must therefore be exploiting genuine non-linear capacity.

5.2.3 Hypothesis

We expect the baseline references to be unable to drive their training error to zero, since the decision rules are non-linear and not separable by a hyperplane or a shallow tree. We expect the proposed model, with either kernel, to fit the training set perfectly once P is large enough, while generalising no better than chance, exhibiting the train–test gap characteristic of an overfitting-capable model. We further expect the picture to be consistent across all three datasets, indicating that the capacity is a property of the architecture rather than of a particular problem.

5.2.4 Results

The outcome is consistent across all three datasets and both kernels, and is summarised in Table 2. The baseline references cannot fit the training set: the linear SVM reaches only 0.57–0.61 training accuracy and the depth-three tree 0.56–0.61,

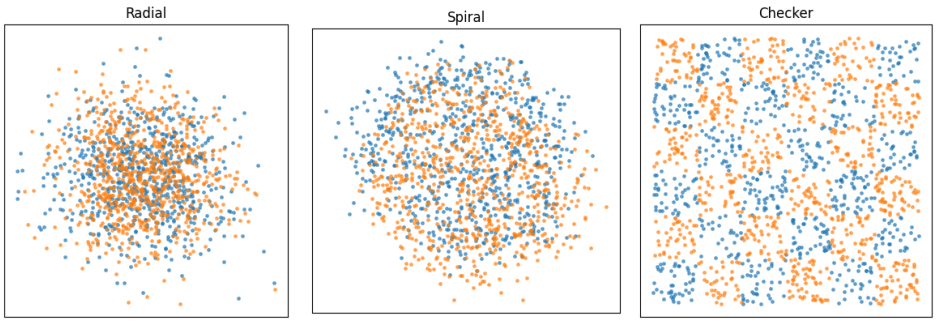

Figure 8: Synthetic datasets used to evaluate non-linear fitting capacity. Each dataset is generated in a low-dimensional informative subspace embedded in a 50-dimensional feature space with Gaussian noise. Radial exhibits a curved multi-region boundary induced by a radial threshold and feature interaction; spiral consists of two interleaving nonlinear manifolds; and checker represents a high-frequency parity structure over a discretized two-dimensional grid. The visualization shows the underlying two-dimensional informative structure used to generate labels, while learning is performed in the full high-dimensional noisy embedding.

both far from fitting the data. The proposed model, by contrast, climbs steadily with P, from roughly 0.60–0.63 at $P = 100$ to about 0.76–0.78 at $P = 500$, reaching a perfect 1.000 training accuracy at $P \geq 1000$. This holds for the arc-cosine and the RBF kernel alike. Throughout, test accuracy remains close to chance (between 0.48 and 0.58 on every configuration), so the perfect training fit reflects pure memorisation rather than any generalisation. The gap between perfect training accuracy and chance-level test accuracy is precisely the overfitting that the regularising mechanisms of the architecture are later required to control.

5.2.5 Discussion

The experiment establishes three things relevant to the rest of the chapter. First, the random-feature map endows the architecture with non-linear capacity that the linear SVM and the shallow tree lack; the proposed model fits boundaries, spirals, and checkerboards that the baseline references cannot, consistent with the arc-cosine map's interpretation as an infinite-width Rectified Linear Unit (ReLU) layer (Section 4.2). Second, this capacity is governed by the feature dimension P: training accuracy rises smoothly with P and saturates at perfect memorisation, and the same behaviour appears for the RBF kernel, showing that the capacity is a property of the random-feature mechanism rather than of one specific kernel. Third, and most importantly for what follows, it fixes the baseline against which the width and depth mechanisms must be read: the architecture begins as a model that can overfit, supplied by P, and the role of m and L, studied next, is to convert that capacity into generalisation rather than to create it. When, in the experiments below, multiple SVMs or additional layers change test accuracy, this is to be understood as regularisation of an already

Model	P	Train accuracy	Test accuracy
radial			
Linear SVM	—	0.577 ± 0.006	0.482 ± 0.023
Decision tree (d. 3)	—	0.577 ± 0.014	0.491 ± 0.006
Arc-cosine	500	0.759 ± 0.007	0.493 ± 0.008
Arc-cosine	1000	1.000 ± 0.000	0.511 ± 0.016
RBF	1000	1.000 ± 0.000	0.509 ± 0.019
spiral			
Linear SVM	—	0.612 ± 0.020	0.576 ± 0.015
Decision tree (d. 3)	—	0.606 ± 0.008	0.569 ± 0.006
Arc-cosine	1000	1.000 ± 0.000	0.510 ± 0.019
RBF	1000	1.000 ± 0.000	0.533 ± 0.007
checker			
Linear SVM	—	0.570 ± 0.003	0.505 ± 0.017
Decision tree (d. 3)	—	0.560 ± 0.014	0.493 ± 0.009
Arc-cosine	1000	1.000 ± 0.000	0.521 ± 0.008
RBF	1000	1.000 ± 0.000	0.526 ± 0.021

Table 2: Overfitting capacity on three hard non-linear problems ($n = 1500$, $d = 50$, few informative features). Mean $\pm$ standard deviation over three seeds. The baseline references cannot fit the training set; the proposed model reaches perfect training accuracy with either kernel once $P \geq 1000$, while test accuracy stays near chance. Intermediate P values for the proposed model are shown for radial to illustrate the climb; the full grid is in Appendix A.

overfitting-capable model, not as the lifting of a representational ceiling.

5.3 Experiment 2: Injecting Diversity into the Parallel SVMs

5.3.1 Methodology

Each block places m linear SVMs in parallel, and the columns of the projection matrix $\mathbf{W}$ are their weight vectors (Section 4.1). If the m SVMs were trained identically they would produce identical weight vectors, and the projection would collapse to rank one; the block must therefore inject diversity among them. Two classical mechanisms for diversifying an ensemble of learners are available, and they are orthogonal. The first, originating with bagging [14], varies the instances each learner sees by partitioning or resampling the rows of the data. The second, the random subspace method [32], varies the features each learner sees by sampling the columns. This experiment asks which axis of diversity (instance or feature) yields the most effective projection, and whether the answer depends on the data regime.

We compare four diversity strategies, named for the axis they vary, together with their combination. Writing n for the number of training samples and d for the input

dimension, and recalling that the block contains m SVMs:

- **Instance Partitioning** trains each SVM on a disjoint n/m partition of the rows, using all d features. Diversity comes from the instances alone.
- **Feature Subsampling** gives each SVM all n rows but a random subset of $\sqrt{d}$ features, the canonical random subspace method [32].
- **Feature Partitioning** gives each SVM all n rows but a disjoint block of d/m features, so the SVMs jointly tile the input space.
- **Joint Partitioning** combines a data partition with a feature subset (in the $\sqrt{d}$ and d/m forms), and so varies both axes at once.

All strategies share the same full per-SVM feature budget $P = 1000$, so they differ only in what data each SVM sees, never in approximation capacity. We verify this control at the end of the section.

5.3.2 Experimental Design

We evaluate every strategy at a fixed width $m = 10$ on six representative datasets that span a wide range of input dimension and sample size: MNIST ($d = 784$), Fashion-MNIST ($d = 784$), and Gisette ($d = 5000$) at the high-dimensional end, and HIGGS ($d = 28$), CoverType ($d = 54$), and Optdigits ($d = 64$) at the lower-dimensional end, each swept over a range of training sizes. The panel is balanced between the two ends so that the overall ranking is not driven by an unequal number of datasets favouring one strategy; a regime-dependent result must then emerge from the data rather than from the composition of the benchmark.

The comparison is held at $m = 10$, not larger, because Feature Partitioning interacts with width: at $m = 50$, on the lower-dimensional datasets the d/m block allocates very few features per SVM, which collapses that strategy and lets Feature Subsampling win by default. The effect of m itself is studied separately in Experiment 3; reading the present comparison at $m = 10$ keeps all strategies comparably conditioned. To assess significance across the heterogeneous datasets we use the Friedman test followed by the Nemenyi post-hoc procedure [20], treating each (dataset, n) combination as a block and the strategies as the treatments. Finally, to probe how diversity acts, we record the mean pairwise cosine similarity of the rows of $\mathbf{W}$ in each block and relate it to accuracy.

5.3.3 Hypothesis

Each diversity mechanism trades a reduction in redundancy among the SVMs against a reduction in the information available to each one, and the balance of that trade

should depend on the data regime. Feature-based diversity withholds columns from each SVM: this should be affordable, and therefore effective, when d is large enough that the retained features still describe the problem, but harmful when d is small and each removed column is costly. Instance-based diversity withholds rows: this should be affordable when n is large relative to the difficulty of the problem, so that an n/m partition still trains a competent SVM, but harmful when data is scarce. On these grounds we predict a regime dependence rather than a single dominant strategy, with feature-based diversity favoured at large d and instance-based diversity favoured at small d with ample n.

Joint Partitioning is the case where we are least certain of the direction. Varying both axes at once compounds the information loss, and our prior expectation is that this may make it the weakest strategy. We note, however, a regime in which it might instead help: when both d and n are large, each SVM could tolerate the loss of some rows and some columns at once while gaining diversity along both axes, in which case Joint Partitioning could match the single-axis strategies.

Finally, the correlation between decorrelation and accuracy tests whether the architecture behaves as the stacker it is constructed to be. By design (Section 4.1), the downstream SVM learns to recombine the projected columns, so decorrelation among the SVMs is not essential in itself: it shapes the projection, but accuracy depends on what the downstream SVM makes of it. We therefore expect that, in general, accuracy will not rise simply with decorrelation. The opposite pattern characterises an ensemble averager, whose SVMs act as independent voters and whose accuracy increases as they decorrelate.

5.3.4 Results

Across the 16 blocks at $m = 10$ (combinations of dataset and n), the Friedman test rejects the hypothesis that the strategies rank equally ($\chi^2 = 46.9$, $p < 10^{-8}$). The average ranks are reported in Table 3 and the corresponding Nemenyi critical-difference diagram in Figure 9; the per-dataset behaviour as a function of n is shown in Figure 10.

Diversity strategy	Average rank
Feature Partitioning (d/m)	1.63
Instance Partitioning	1.81
Joint Partitioning (d/m)	3.13
Feature Subsampling ($\sqrt{d}$)	3.50
Joint Partitioning ($\sqrt{d}$)	4.94

Table 3: Average ranks of the diversity strategies at $m = 10$ over the 16 (dataset, n) blocks (lower is better). A Friedman test rejects rank equality ($\chi^2 = 46.9$, $p < 10^{-8}$); the Nemenyi critical difference at $\alpha = 0.05$ is CD = 1.53.

The ranking shows the two single-axis strategies clearly ahead. Feature Partitioning (1.63) and Instance Partitioning (1.81) lead, are not significantly different from each other (Δ rank $= 0.19 \ll$ CD), and each is significantly better than Feature Subsampling and than the worst strategy, Joint Partitioning ($\sqrt{d}$). Joint Partitioning (d/m) sits just behind the leaders at rank 3.13: its gap to Feature Partitioning (1.50) is just below the critical difference of 1.53, so it is not formally separated from the leading group, though it falls close to the boundary. The remaining strategies form a lower tier, with Joint Partitioning ($\sqrt{d}$) significantly the worst of all. Subsampling features at random, rather than partitioning them, and varying both axes in the $\sqrt{d}$ form remove more shared signal than the projection can afford; the structured single-axis strategies are preferable. The prior conjecture that Joint Partitioning might match the single-axis strategies when both d and n are large is only weakly supported: the d/m form approaches the leaders but does not reach them, and the $\sqrt{d}$ form is the worst strategy throughout.

Although the two leading strategies are tied overall, which of them wins is governed cleanly by the data regime, and this is the central practical finding. Figure 10 and Appendix C report the accuracy of each strategy by training size. Feature Partitioning wins throughout on the high-dimensional datasets: it is the best strategy at every training size on MNIST (from 0.893 at $n = 1000$ to 0.971 at $n = 50{,}000$), Fashion-MNIST (0.834 to 0.892), and Gisette (0.967 and 0.971), with no crossover at any size. Instance Partitioning wins on the lower-dimensional, large-sample datasets: where both strategies are available, it is the best on HIGGS and CoverType for every $n \geq 5000$. The clearest within-dataset evidence comes from the intermediate-dimensional Optdigits, where both strategies are present across the whole sweep and the lead crosses over from Feature Partitioning at $n \leq 1000$ to Instance Partitioning at $n \geq 2000$. A transition occurring within one dataset, holding all else fixed, indicates that the boundary is governed by the data regime (d relative to n) rather than by differences between datasets.

5.3.5 Discussion

The two leading strategies, and the clean regime split between them, confirm the hypothesis: diversity is best injected along a single axis, and the data regime selects which. When the input dimension is high, partitioning the features gives each SVM a distinct yet still informative view, and Feature Partitioning leads; when the dimension is lower and data is plentiful, withholding features is too costly and partitioning the instances is the better source of diversity, so Instance Partitioning leads. The strategies that vary both axes at once, or that subsample features at random rather than partitioning them, underperform: the structured, single-axis strategies preserve more of the signal each SVM needs.

The relationship between decorrelation and accuracy clarifies the underlying mechanism, and here we make an observation rather than a primary claim. We

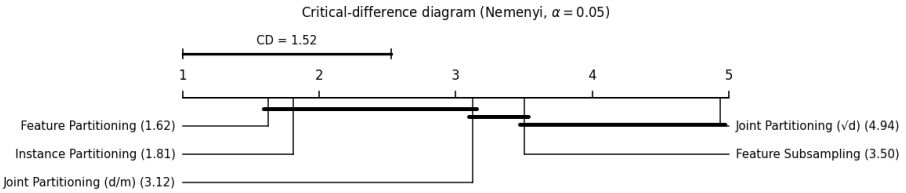

Figure 9: Critical-difference diagram (Nemenyi, $\alpha = 0.05$) for the diversity strategies at $m = 10$. Strategies are placed on the average-rank axis (best to the left); a bar joins strategies whose ranks differ by less than the critical difference and are therefore not statistically distinguishable. Feature Partitioning and Instance Partitioning form the leading group; Joint Partitioning (d/m) falls just outside it, at a gap from the leaders only marginally below the critical difference, so although not formally part of the leading group it is best read as close to it. Joint Partitioning ($\sqrt{d}$) ranks worst.

measured, per dataset, the Pearson correlation between the mean cosine similarity of the rows of $\mathbf{W}$ and test accuracy across the $m > 1$ configurations (Table 4). In the majority of cases this correlation is weak and statistically insignificant ($|r| \leq 0.15$, $p > 0.29$ on four of the five datasets for which the diagnostic is available), indicating that decorrelation does not, by itself, improve accuracy. This is consistent with the architecture operating as a stacker, in which the downstream SVM learns to recombine the projected features, rather than as a pure ensemble averager, for which maximal decorrelation would be the objective. One dataset (Gisette) is an exception, showing a strong positive correlation; we note it without drawing a general conclusion, as a correlation pooled across datasets is in any case confounded by their differing accuracy levels and cosine ranges. Decorrelation is therefore best treated as a diagnostic reported alongside accuracy, not as a quantity to be maximised.

Dataset	# configs	Pearson r	p
CoverType	48	-0.100	0.499
FashionMNIST	30	$+0.003$	0.987
Optdigits	44	$+0.126$	0.415
MNIST	52	$+0.149$	0.293
Gisette	20	$+0.879$	< 0.001

Table 4: Pearson correlation, per dataset, between the mean cosine similarity of the rows of $\mathbf{W}$ and test accuracy across the $m > 1$ configurations.

The practical conclusion for the remainder of the study is that the feeding strategy should be chosen by regime: partition the features when d is large, and partition the instances when d is small with ample n. Experiment 3 adopts this regime-dependent

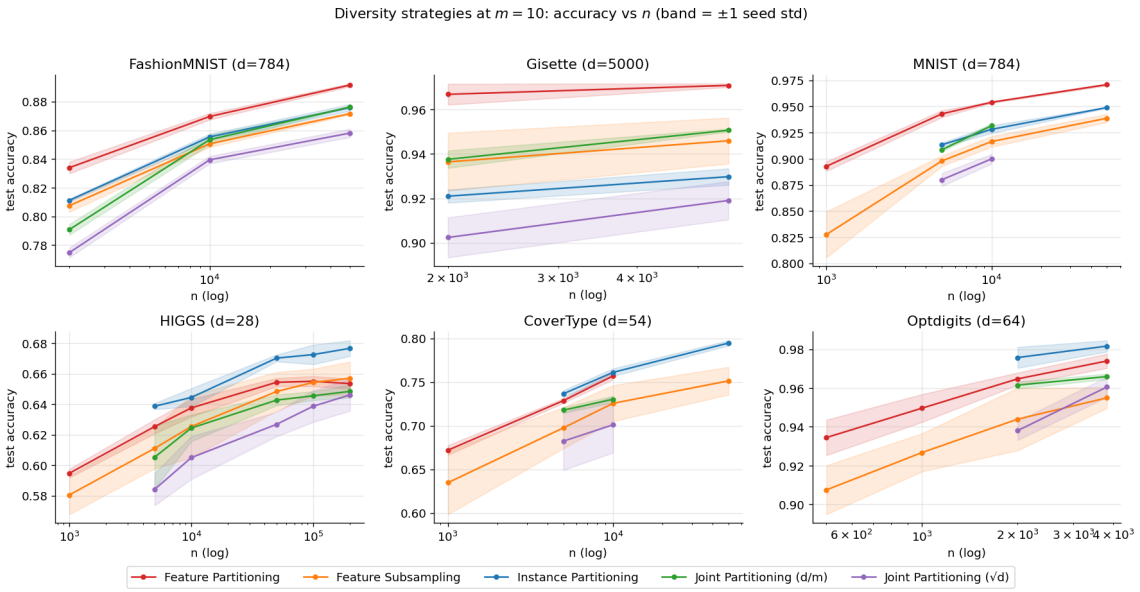

Figure 10: Test accuracy as a function of training-set size n for each diversity strategy at $m = 10$, per dataset. Shaded bands denote ±1 seed standard deviation. Some strategies are absent at the smallest n: Instance and Joint Partitioning split the n training rows among the $m = 10$ SVMs, and are not run when this would leave fewer than 200 samples per SVM, which is why they have no point at the smallest size on HIGGS and CoverType. On the high-dimensional datasets (MNIST, Fashion-MNIST, Gisette) Feature Partitioning leads at every n; on the lower-dimensional, large-sample datasets (HIGGS, CoverType) and the intermediate Optdigits the lead crosses over to Instance Partitioning as n grows.

choice rather than fixing a single strategy throughout.

Feature-Budget Control. The comparison above held the per-SVM feature budget fixed at the full $P = 1000$ across all strategies, so that diversity, not approximation capacity, is the only thing that varies. To justify this we compared it against an earlier scheme allocating only P/m features per SVM. At fixed dataset, strategy, and m, the full-P budget matches or exceeds the P/m budget at every comparable configuration, and the gap widens with m, as expected since P/m shrinks the budget in proportion to the width. This confirms that the budget is a genuine confound; the P/m configurations are reported in Appendix B and excluded from the ranking.

5.3.5.1 A Geometric View: Why Diversity Prevents Rank Collapse

The decorrelation diagnostic of the previous subsection is summarised most directly in the geometry of the projected representation itself. Recall that a block emits

$\mathbf{X}_{\text{next}} = \mathbf{\Phi W}$, whose columns are the decision projections of the m SVMs. If those SVMs discriminate along the same direction, the columns are collinear and $\mathbf{X}_{\text{next}}$ is effectively one-dimensional; if they discriminate along genuinely different directions, the representation retains higher rank and the downstream block has richer structure to recombine. To make this concrete we train a three-block instance of the architecture ($m = 10$) on two-dimensional problems where the representation can be drawn directly, and compare the worst feeding strategy against the two winners from the ranking above.

Figure 11 shows the result on a two-class spiral, a problem that a single SVM block cannot separate and that therefore exercises the projection. Under the same-data strategy that varies only the regularisation constant C, the ten boundaries of every block coincide into a single curve, and the representation entering the final classifier collapses onto a one-dimensional fold: the points lie on a single axis, exactly the rank-one outcome anticipated when the SVMs are not diversified. Under Instance Partitioning and Feature Partitioning the picture is qualitatively different. The block boundaries fan out into distinct directions, and the projected representation occupies two dimensions rather than one, with the classes distributed across the plane instead of pinned to a line. The accuracy figures of this subsection confirm that this retained structure is useful and not merely noise: the same-C strategy gains nothing from depth on this problem, whereas the two partitioning strategies improve markedly as blocks are added. The figure may be read as evidence that diversity prevents collapse, with the accuracy carrying the separate claim that the preserved structure is informative.

The Multiclass Case Diversifies Itself. The collapse above is a property of binary problems, and it is worth stating why. When the SVMs are trained one-against-rest on K classes, each emits $K - 1$ decision columns rather than one, so a single block already produces a projection of width $m(K-1)$ whose columns span a label-relevant subspace of dimension up to $K-1$. For $K = 2$ this subspace is a single axis, and without an external source of diversity among the SVMs the projection collapses onto it; this is precisely why a binary block depends on the feeding strategy to avoid rank one. For $K > 2$ the label geometry supplies the diversity for free: the $K - 1$ class directions are distinct by construction, so the projection cannot collapse to a single axis regardless of how the SVMs are fed. This distinction is not merely cosmetic. It is the reason, taken up in Experiment 3, that depth and width behave differently on binary and multiclass problems: the binary case must import the multi-dimensional structure that the multiclass case already possesses. We emphasise that the collapse is driven by the dimension of the label-relevant subspace, not by any sameness of the SVMs themselves; under instance or feature partitioning the binary weight vectors are genuinely distinct, yet they still ultimately separate the same one-dimensional target, which is what pins the representation to a single axis.

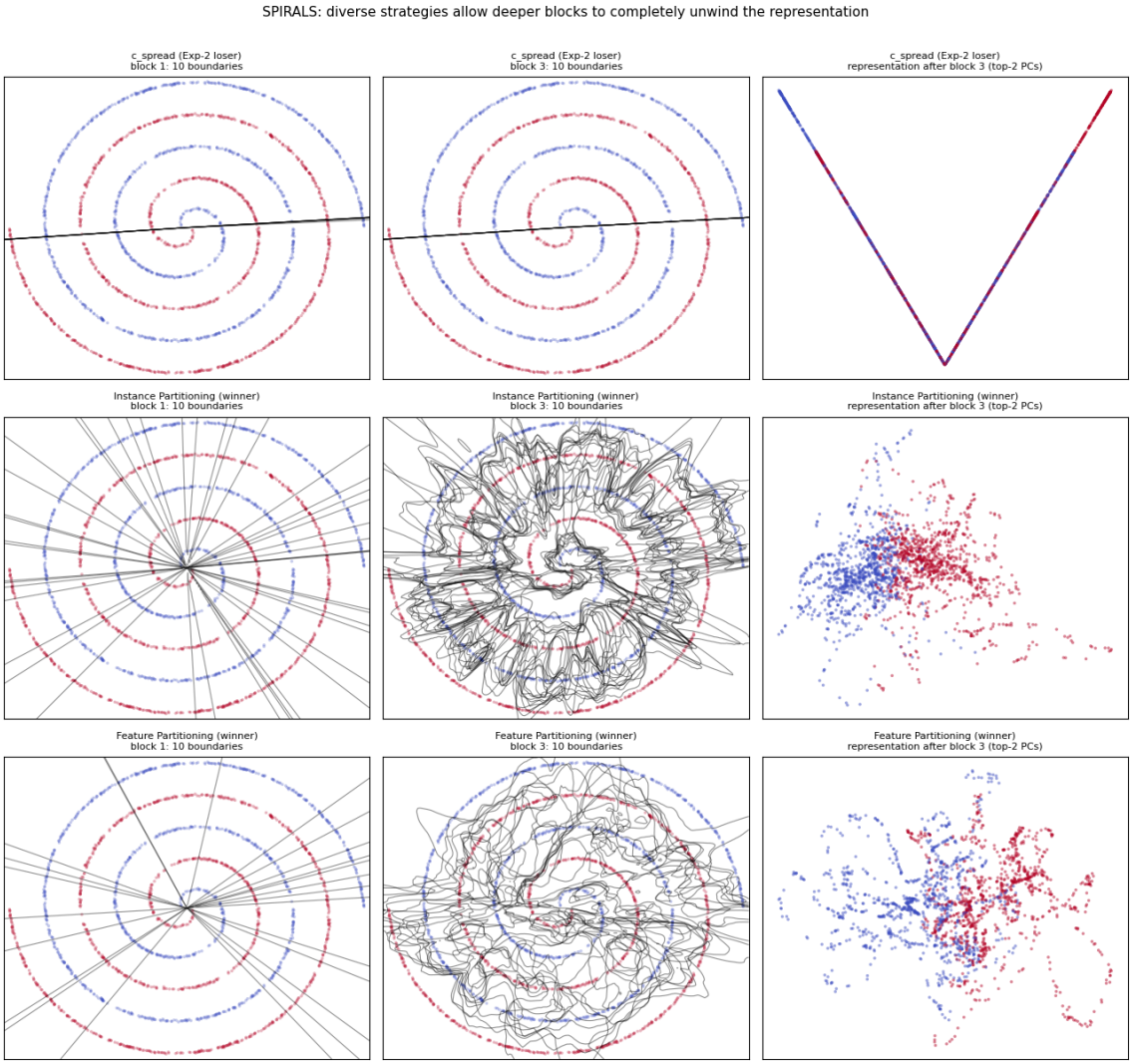

Figure 11: Geometric effect of the feeding strategy on a two-class spiral, $m = 10$ SVMs per block, three blocks. Columns show the ten SVM boundaries at the first block, the ten boundaries at the third block, and the representation entering the final classifier (projected onto its two leading principal components). Top row: varying only C on shared data; the boundaries coincide and the representation collapses onto a single one-dimensional fold. Middle and bottom rows: Instance Partitioning and Feature Partitioning; the boundaries stay diverse and the representation retains two-dimensional structure.

5.4 Experiment 3: Width and Depth in the Large-Sample Complex Regime

5.4.1 Methodology

This experiment studies the two structural parameters of the architecture, the block width m (the number of parallel SVMs per block) and the depth L (the number of

stacked blocks), and asks where each provides value. The question is framed by the regime the architecture is designed for: data complex enough to demand a multi-layer model, at a sample size large enough that an exact kernel SVM is intractable. On easy or low-sample problems an exact kernel SVM is both applicable and strong, and we do not seek to compete there.

The grid sweeps width $m \in \{20, 50, 70\}$ against depth $L \in \{1, 2, 3, 4, 5\}$. Each SVM is fed by the regime-appropriate strategy established in Experiment 2: feature partitioning when the input dimension is large, and instance partitioning when it is small with ample data. This choice is not incidental. Section 5.3.5.1 showed that a binary block collapses to a one-dimensional projection unless the SVMs are diversified; the feeding strategy is what supplies that diversity, and depth can only act on a representation that has not already collapsed.

We record, for both the training and the test set, the accuracy and the macro-averaged F_1 score. We report macro-F_1 as the headline metric on the imbalanced datasets and accuracy on the balanced ones.

5.4.2 Experimental Design

The two structural parameters give rise to three empirical questions, and the experiment is designed to separate them:

- whether adding layers (L) adds capacity to fit more complex data;
- whether increasing the width (m) supplies subsequent layers with a richer representation, raising the accuracy that depth can reach;
- whether m acts as a regulariser, with its effect depending on the sample size.

The datasets are chosen so that each instantiates the regime under test: one non-linear layer is insufficient and an exact kernel SVM is intractable. To identify such datasets we screened candidates against a criterion fixed before any architectural experiment was run: a dataset is admitted only if a strong ANN clearly outperforms the RBF kernel at a feasible sample size. The margin between the two is the headroom the architecture would have to recover for depth or width to matter; its absence means the problem is already solved by a shallow kernel. Where an exact RBF SVM was intractable, it was replaced by its RFF approximation, validated against the exact kernel at a smaller size.

Table 5 reports the datasets. Three are real and two are synthetic. The real datasets share a defining property: their labels are near-deterministic functions of the inputs (a hand ranking, a terrain class, an optimal-play depth) with low label noise but a highly non-smooth decision boundary, which a locally smooth RBF kernel cannot capture but a deeper model can. The two synthetic datasets, a binary two-spiral and a binary checkerboard, instantiate the same property under controlled conditions.

Dataset	Type	d	n	ANN	RBF	Gap (%)
Poker Hand	real, multiclass	10	60,000	0.999	0.562†	+43.7
CoverType	real, multiclass	98	60,000	0.902	0.786	+11.6
Chess (KRK)	real, multiclass	23	14,444	0.833	0.597	+23.6
Spirals	synthetic, binary	2	10,000	1.000	0.863	+13.7
Checkerboard	synthetic, binary	2	10,000	0.978	0.942	+3.6

Table 5: Dataset screen for Experiment 3. The gap is the accuracy margin of a strong ANN over the RBF kernel at the screening size.

The screen is conducted at a fixed, feasible size; Experiment 3 then evaluates at each dataset's largest available pool (Table **??**). The RBF reference therefore differs between the two tables, as its accuracy degrades once n grows beyond the exact-kernel regime, a degradation that is itself part of the motivation for a scalable alternative.

5.4.3 Hypothesis

We expect adding layers (L) to improve the test metric on the datasets where one nonlinear layer is insufficient. From Section 5.3.5.1 we further expect depth to work more reliably on multiclass than on binary problems, because a multiclass block already emits a multi-dimensional projection for depth to act on whereas a binary block is one-dimensional unless externally diversified.

We expect increasing the width (m) to raise the test metric that depth can reach, with diminishing returns once the projection is wide enough to carry the label-relevant structure.

Finally, we expect m to act as a regulariser whose value depends on the sample size: beneficial when n is small relative to the problem's difficulty, and of diminishing value as n grows.

5.4.4 Results

Table 6 reports, per dataset at its largest sample size, the flat arc-cosine baseline, the RBF reference, the best architecture over the grid, and the depth gain.

Where depth helps. The architecture climbs substantially with depth on three datasets, shown in Figure 12. On Spirals the test accuracy goes from 0.592 at L=1 to 1.000 at L=3 with m=50, a relative gain of +68.7%, matching the ANN and beating the RBF reference (0.735) by +36.1% relative; the seed-standard-deviation bands tighten as L grows, indicating that depth reduces variance on top of raising the mean. On Checkerboard the architecture rises from 0.767 to 0.961, a relative gain of +25.3%, again beating the RBF reference. Chess (KRK) shows a smaller gain of

Dataset	Metric	Flat	RBF	L=1	Best	Best (m, L)	Depth gain
Spirals	test acc	0.590	0.735	0.592	1.000	$(50, 3)$	+68.7%
Checkerboard	test acc	0.766	0.736	0.767	0.961	$(70, 5)$	+25.3%
Chess (KRK)	test acc	0.534	0.597	0.543	0.584	$(70, 4)$	+7.7%
CoverType	test acc	0.786	0.723	0.767	0.768	$(70, 2)$	+0.0%
Poker Hand	macro F1	0.191	0.242	0.252	0.252	$(20, 1)$	0.0%

Table 6: Per-dataset summary at the largest sample size. *Flat* is the flat arc-cosine SVM; *RBF* is the exact RBF SVM where feasible (Chess (KRK)) and its RFF approximation otherwise. L=1 is the architecture at unit depth using the best width. *Best* is the highest test value reached over the (m, L) grid. Depth gain is the relative change from L=1 to *Best* at the best width, $(\text{Best} - L_1)/L_1 \times 100$. For Poker Hand the best configuration is itself L=1, so depth yields no gain; the degradation of test performance with added depth is reported in the text.

+7.7% relative, from 0.543 to 0.584, approaching but not reaching the exact-RBF value of 0.597.

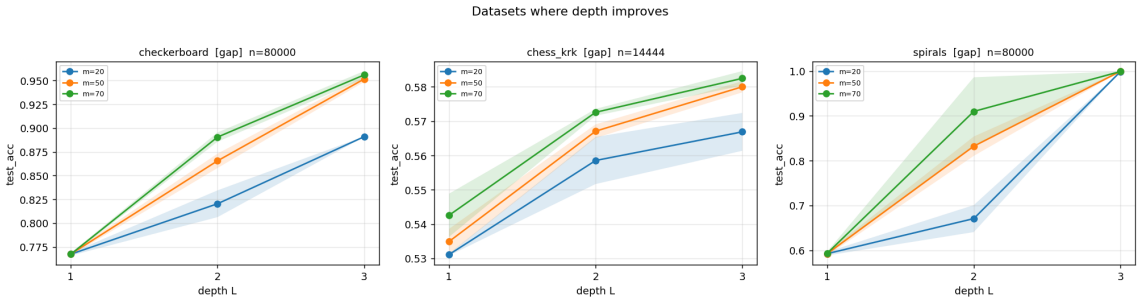

Figure 12: Test metric versus depth L for the datasets where the architecture rises with depth, at each dataset's largest sample size. Solid lines are seed means at each width m; bands are ± 1 seed standard deviation.

Where depth does not help. Figure 13 shows the two datasets where depth is flat or counterproductive. Covertype is essentially flat with depth: at the best width the test accuracy moves only from 0.767 at L=1 to 0.768 at L=2 (+0.0% relative) and is flat or slightly declining thereafter. On Poker Hand, the best test macro-F1 is attained at L=1, and depth produces no gain across $L = 1, \ldots, 5$ for all three widths.

The Poker result is particularly informative because the failure of depth here is active rather than passive. Across all (m, L) configurations at n=100,000, training macro-F_1 climbed from 0.39 at L=1 to 0.66 at L=5 (a +69% relative increase on the training set), while test macro-F_1 stayed within ± 0.02 of 0.19. The train–test gap

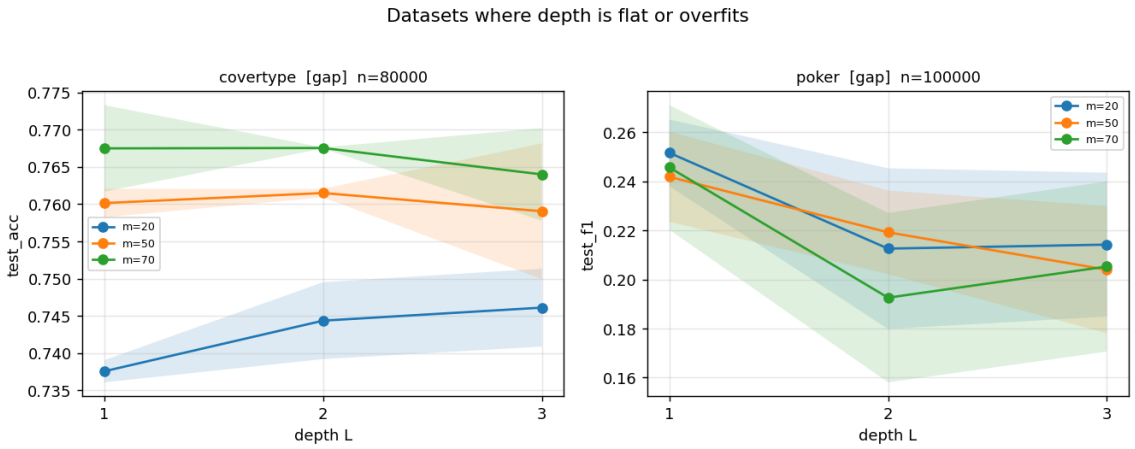

Figure 13: Test metric versus depth L for the datasets where depth produced no clear improvement. Line conventions as in Figure 12.

therefore widened from 0.20 to 0.47 across the depth sweep. A targeted diagnostic at n=40,000, m=50 (illustrated in Figure 14, right panel) confirmed that this is overfitting and not the rank-collapse failure of Section 5.3.5.1: the mean pairwise $|\cos|$ of the per-block weight vectors stayed between 0.035 and 0.066 across layers, and the stable rank of the inter-layer representation remained between 3.8 and 10.4. The added blocks are therefore producing diverse, multi-dimensional projections; what they cannot produce is projections that generalise.

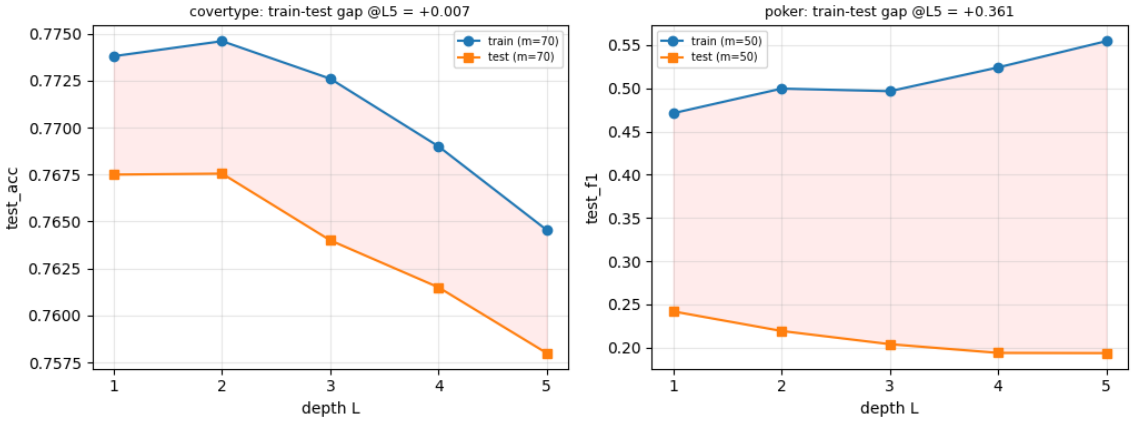

Figure 14: Training and test performance across depth L for the Covertype (left) and Poker (right) datasets, highlighting the train–test gap (shaded red). Covertype displays a passive failure where both training and test accuracies decline simultaneously with added layers. In contrast, Poker demonstrates active overfitting: training macro-F_1 steadily increases while test macro-F_1 degrades.

Width. Figure 15 reports the test metric versus width m at the best depth L for each (dataset, n) pair. The picture is more uniform than for depth: where the architecture has headroom, wider blocks help, and the effect is monotone in m on every dataset except Spirals, which is already saturated.

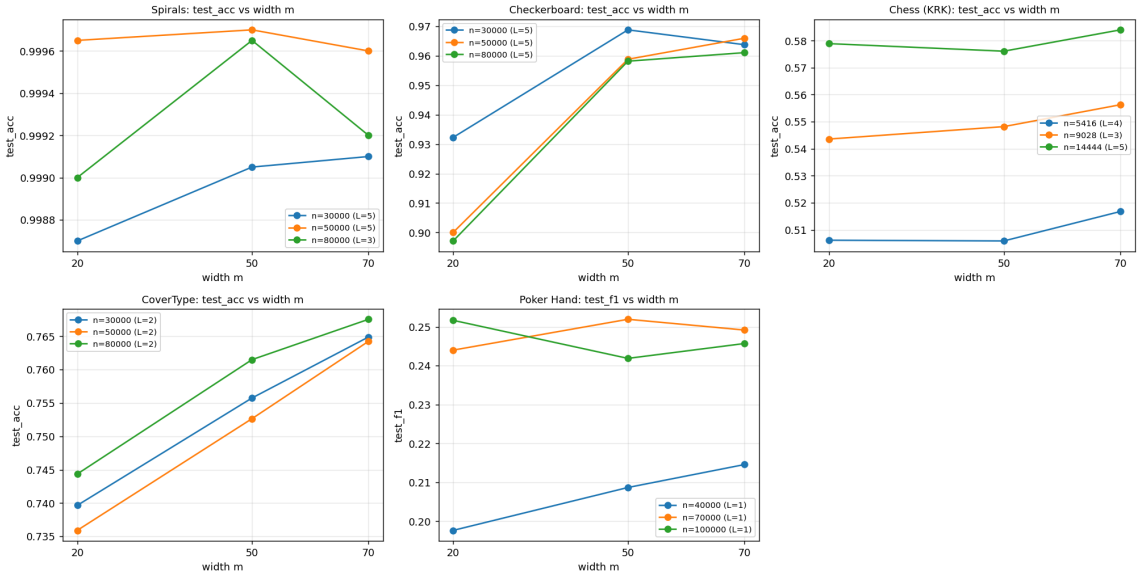

Figure 15: Test metric versus width m, at the best depth L for each dataset and training size n.

The clearest width effect appears on Checkerboard and CoverType. On Checkerboard the test accuracy rises from about 0.90 at m=20 to 0.96 at m=70 across all three sample sizes, a relative gain of roughly +7%, with the three n curves nearly coincident. CoverType shows the same shape, climbing from 0.74 to 0.77 (+3.6% relative at n=80,000) with the curves ordered cleanly by sample size. In both cases the gain is steepest from m=20 to m=50 and flatter from m=50 to m=70, the diminishing return predicted once the projection is wide enough to carry the label-relevant structure.

On Spirals the architecture is already at ceiling: every configuration exceeds 0.998 test accuracy, and the variation across m is within seed noise. No width conclusion can be drawn here beyond the absence of degradation.

The two remaining datasets restrict the width effect to the smallest sample size. On Chess (KRK) width is essentially flat at n=9,028 and n=14,444 but rises at the smallest pool (n=5,416), from 0.506 at m=20 to 0.517 at m=70. Poker Hand shows the same pattern: macro-F_1 is flat in m at n=70,000 and n=100,000 but rises at n=40,000, from 0.198 to 0.214 (+8.1% relative). On both datasets the larger pools gain nothing from width, while the smallest pool does.

5.4.5 Discussion

The experiment separates cleanly into the three questions it was designed to answer, and the answers are not the same for depth and width.

Depth supplies capacity, but only where the data has the structure to use it. The architecture climbs strongly with depth on Spirals, Checkerboard, and Chess (KRK), and the gain is largest on the synthetic problems whose non-smooth boundaries a single nonlinear layer cannot represent. The hypothesised split between multiclass and binary problems is not borne out in the simple form predicted: depth helps the binary synthetic datasets most of all, because the feeding strategy supplies the diversity that Section 5.3.5.1 showed a binary block needs, and once that diversity is present the binary projection is no longer the limiting factor. The relevant distinction is therefore not binary against multiclass but whether the representation has collapsed; a diversified binary block behaves like a multiclass one.

The two datasets where depth fails do so for different reasons, and the distinction matters. CoverType is a passive failure: training and test metrics decline together as layers are added, so the deeper model is not learning the training set better, and the architecture is simply past the point where extra blocks help. Poker Hand is an active failure: training macro-F_1 rises with depth while test macro-F_1 does not, and the diagnostic confirms this is overfitting rather than collapse, since the inter-layer projections remain diverse and high-rank. The added blocks fit more of the training labels but produce nothing that transfers. This is the expected behaviour of a model whose smooth arc-cosine features are mismatched to the combinatorial rank logic the Poker labels encode: capacity exists, but it cannot be aimed at the right function. The screen in Table 5 predicts headroom but not whether the architecture can recover it, and Poker is the case where it cannot.

Width acts more simply. Where the architecture has headroom and has not collapsed, wider blocks raise the test metric monotonically, with diminishing returns from m=50 onward; this matches the prediction that m raises the accuracy depth can reach until the projection is wide enough to carry the label structure. The hypothesis that m acts as a regulariser whose value depends on sample size is supported only at the margin: on Chess (KRK) and Poker Hand width helps at the smallest pool and not at the larger ones, which is the predicted direction, but the effect is small and absent on the datasets that are not sample-limited. We therefore read width as primarily a representation-richness parameter rather than a regulariser, with a sample-size interaction visible only where data is scarce relative to the problem.

Taken together, depth and width are complementary but not interchangeable. Depth converts the per-block capacity into a deeper composed function and is the parameter that closes the gap to the ANN on the structured problems; width broadens each projection and lifts the level that depth can reach. Neither can rescue a dataset whose target is mismatched to the kernel, as Poker shows: there the architecture has both diverse projections and growing training fit, and still does not

generalise.

5.5 Experiment 4: Benchmark Against Standard and Deep-Kernel Baselines

5.5.1 Methodology

This experiment compares the proposed architecture against several well-known methods on tabular data in the classification setting. We select five baseline methods that span the linear, ensemble, shallow-kernel, deep-neural, and exact-kernel families.

The linear SVM (with squared hinge loss, evaluated via a dual-free solver) acts as a baseline sanity floor. The gradient boosting classifier (200 iterations, library defaults) serves as the state-of-the-art strong baseline for tabular data. The shallow RFF SVM isolates the contribution of depth from that of the random-feature map, using a flat single-layer RFF map with a budget of P=1000 features followed by a linear classifier trained via Stochastic Gradient Descent (SGD) with a hinge loss. The multi-layer perceptron consists of three hidden layers of 200 units each with ReLU activations, trained via backpropagation. Finally, the exact RBF SVM (C=10, with γ determined via the scale heuristic) marks the exact kernel boundary and is computed exclusively when the effective training set size satisfies $n \leq 70{,}000$.

To ensure a scientifically rigorous and coherent evaluation, the proposed architecture is tuned **exactly once per dataset** rather than per seed. This locks in a single global configuration for each problem, preventing model-blending artifacts across experimental trials. The optimal architecture is selected by maximizing the mean validation accuracy across three independent internal validation splits (80/20 train/validation) drawn from a training subset capped at 10,000 samples for computational efficiency. The optimization grid exhaustively searches across the following hyperparameter combinations:

- **Depth (L):** $L \in \{1, 2, 3, 4, 5\}$ to explicitly test shallow vs. deep regimes.
- **Width (m):** $m \in \{50, 70\}$ blocks per layer.
- **Kernel Family:** Base block kernels chosen from $\{\text{arc-cosine}, \text{RBF}\}$.
- **Kernel Degree (n):** When the arc-cosine kernel is selected, its degree parameter is actively optimized over $n \in \{0, 1, 2\}$.
- **Feeding Strategy:** The diversity mapping mode is explicitly tuned as a categorical hyperparameter over $\{\text{disjoint}, \text{disjoint\_featpart}\}$, completely discarding static feature-count heuristics.

The total random-feature budget per block is fixed at P=1000. Once the single optimal configuration is selected via the internal multi-split validation mean, it is

locked and executed across all three outer experimental seeds using the full training split. Baselines use their library defaults to maintain a comparable hyperparameter tuning budget across methods.

For every (dataset, method, seed) cell, we record the training and test evaluation metrics, tracking accuracy and macro-F_1 score.

5.5.2 Experimental Design

The dataset suite covers a wide range of sizes, dimensions, and difficulty profiles. A first group consists of standard tabular benchmarks: `breast_cancer` ($n = 286$, $d = 9$), `ionosphere` ($n = 351$, $d = 34$), `ecoli` ($n = 336$, $d = 7$), `glass` ($n = 214$, $d = 9$), `spambase` ($n = 4{,}601$, $d = 57$), `magic` ($n = 19{,}020$, $d = 11$), `adult` ($n = 48{,}842$, $d = 14$), `bank-marketing` ($n = 45{,}211$, $d = 51$), `jannis` ($n = 83{,}733$, $d = 54$), `higgs` ($n = 98{,}050$, $d = 28$), `miniboone` ($n = 130{,}064$, $d = 50$) and `covertype` ($n = 581{,}012$, $d = 98$). A second group is chosen for properties that degrade shallow or axis-aligned models: `madelon` ($n = 2{,}600$, $d = 500$, with five relevant features embedded in noise dimensions), `letter` ($n = 20{,}000$, $d = 16$, 26-class with highly interleaved boundaries), `mnist` ($n = 70{,}000$, $d = 784$, flattened pixels) and `spirals` ($n = 120{,}000$, $d = 2$, synthetic interlocking class structures).

For each (dataset, seed) we draw a single stratified train/test split with the test set set to 10% of the dataset pool. The training pool is capped at 100,000 samples; on datasets larger than this the excess is discarded uniformly at random. Each method is fitted on the same training partition and evaluated on the same test partition. Four random seeds per cell allow the seed-level variance to be estimated.

6 Conclusion

References

[1] Azizi Abdullah, Remco C Veltkamp, and Marco A Wiering. "An ensemble of deep support vector machines for image categorization". In: *2009 International Conference of Soft Computing and Pattern Recognition*. IEEE. 2009, pp. 301–306.

[2] Joan Acero-Pousa and Lluís A Belanche-Muñoz. "A new approach to multilayer SVMs". In: *ESANN 2025 proceedings, European Symposium on Artificial Neural Networks, Computational Intelligence and Machine Learning*. 2025, pp. 467–472.

[3] AL Afzal and S Asharaf. "Deep multiple multilayer kernel learning in core vector machines". In: *Expert Systems with Applications* 96 (2018), pp. 149–156.

[4] Shun-ichi Amari. "Backpropagation and stochastic gradient descent method". In: *Neurocomputing* 5.4-5 (1993), pp. 185–196.

[5] Arthur Asuncion, David Newman, et al. *UCI machine learning repository*. 2007.

[6] Michael Bain and Arthur Hoff. *Chess (King-Rook vs. King)*. UCI Machine Learning Repository. DOI: https://doi.org/10.24432/C57W2S. 1994.

[7] Barry Becker and Ronny Kohavi. *Adult*. UCI Machine Learning Repository. DOI: https://doi.org/10.24432/C5XW20. 1996.

[8] Bernd Bischl et al. "OpenML: Insights from 10 years and more than a thousand papers". In: *Patterns* 6.7 (2025), p. 101317. DOI: 10.1016/j.patter.2025.101317. URL: https://www.cell.com/patterns/fulltext/S2666-3899(25)00165-5.

[9] Christopher M Bishop. *Pattern recognition and machine learning*. Springer, 2006.

[10] Jock Blackard. *Covertype*. UCI Machine Learning Repository. DOI: https://doi.org/10.24432/C50 1998.

[11] Salomon Bochner. *Vorlesungen über Fouriersche Integrale*. Akademische Verlagsgesellschaft m.b.H., 1932.

[12] R. Bock. *MAGIC Gamma Telescope*. UCI Machine Learning Repository. DOI: https://doi.org/10.24432/C52C8B. 2004.

[13] Bernhard E Boser, Isabelle M Guyon, and Vladimir N Vapnik. "A training algorithm for optimal margin classifiers". In: *Proceedings of the fifth annual workshop on Computational learning theory*. 1992, pp. 144–152.

[14] Leo Breiman. "Bagging predictors". In: *Machine learning* 24.2 (1996), pp. 123–140.

[15] Robert Cattral and Franz Oppacher. *Poker Hand*. UCI Machine Learning Repository. DOI: https://doi.org/10.24432/C5KW38. 2002.

[16] Jair Cervantes et al. “A comprehensive survey on support vector machine classification: Applications, challenges and trends”. In: *Neurocomputing* 408 (2020), pp. 189–215.

[17] Youngmin Cho and Lawrence Saul. “Kernel methods for deep learning”. In: *Advances in neural information processing systems* 22 (2009).

[18] Youngmin Cho and Lawrence K Saul. “Large-margin classification in infinite neural networks”. In: *Neural computation* 22.10 (2010), pp. 2678–2697.

[19] Corinna Cortes and Vladimir Vapnik. “Support-vector networks”. In: *Machine learning* 20.3 (1995), pp. 273–297.

[20] Janez Demšar. “Statistical comparisons of classifiers over multiple data sets”. In: *Journal of Machine learning research* 7.Jan (2006), pp. 1–30.

[21] Li Deng, Xiaodong He, and Jianfeng Gao. “Deep stacking networks for information retrieval”. In: *2013 IEEE International Conference on Acoustics, Speech and Signal Processing*. IEEE. 2013, pp. 3153–3157.

[22] Li Deng and Dong Yu. “Deep convex net: A scalable architecture for speech pattern classification”. In: *Proceedings of the Interspeech*. 2011, pp. 2285–2288.

[23] Manuel Fernández-Delgado et al. “Do we need hundreds of classifiers to solve real world classification problems?” In: *The journal of machine learning research* 15.1 (2014), pp. 3133–3181.

[24] Yulu Gan and Tomaso Poggio. *For hyperbfs agop is a greedy approximation to gradient descent*. Tech. rep. Center for Brains, Minds and Machines (CBMM), 2024.

[25] B. German. *Glass Identification*. UCI Machine Learning Repository. DOI: https://doi.org/10.2443 1987.

[26] Ian Goodfellow et al. *Deep learning*. Vol. 1. 2. MIT press Cambridge, 2016.

[27] Werner H Greub. *Linear algebra*. Vol. 23. Springer Science & Business Media, 2012.

[28] Maarten Grootendorst. “The Kernel Trick”. In: *TowardsDataScience* (2018). Accessed: June 8, 2024. URL: https://towardsdatascience.com/the-kernel-trick-c98cdbcaeb3f.

[29] Isabelle Guyon. *Madelon*. UCI Machine Learning Repository. DOI: https://doi.org/10.24432/C560 2004.

[30] Isabelle Guyon et al. “Analysis of the AutoML Challenge Series 2015–2018”. In: *Automated Machine Learning: Methods, Systems, Challenges*. Ed. by Frank Hutter, Lars Kotthoff, and Joaquin Vanschoren. The Springer Series on Challenges in Machine Learning. Springer, Cham, 2019, pp. 177–219. DOI: 10.1007/978-3-030-05318-5_10.

[31] Chris Hettinger et al. “Forward thinking: Building and training neural networks one layer at a time”. In: *arXiv preprint arXiv:1706.02480* (2017).

[32] Tin Kam Ho. "The random subspace method for constructing decision forests". In: *IEEE transactions on pattern analysis and machine intelligence* 20.8 (1998), pp. 832–844.

[33] Mark Hopkins et al. *Spambase*. UCI Machine Learning Repository. DOI: https://doi.org/10.24432/ 1999.

[34] Arthur Jacot, Franck Gabriel, and Clément Hongler. "Neural tangent kernel: Convergence and generalization in neural networks". In: *Advances in neural information processing systems* 31 (2018).

[35] William Karush. "Minima of functions of several variables with inequalities as side constraints". In: *M. Sc. Dissertation. Dept. of Mathematics, Univ. of Chicago* (1939).

[36] Sangwook Kim, Swathi Kavuri, and Minho Lee. "Deep Network with Support Vector Machines". In: *Neural Information Processing*. Ed. by Minho Lee et al. Berlin, Heidelberg: Springer Berlin Heidelberg, 2013, pp. 458–465. ISBN: 978-3-642-42054-2.

[37] Joseph-Louis Lagrange. "Méthode des multiplicateurs de Lagrange". In: *Journal de l'École Polytechnique* 13 (1811), pp. 141–175.

[38] Yann LeCun, Yoshua Bengio, and Geoffrey Hinton. "Deep learning". In: *nature* 521.7553 (2015), pp. 436–444.

[39] Yann LeCun et al. "Gradient-Based Learning Applied to Document Recognition". In: *Proceedings of the IEEE* 86.11 (1998), pp. 2278–2324. DOI: 10.1109/5.726791.

[40] Jaehoon Lee et al. "Deep neural networks as gaussian processes". In: *arXiv preprint arXiv:1711.00165* (2017).

[41] Jiyoon Lee. *Support Vector Regression*. https://leejiyoon52.github.io/Support-Vecter-Regression/. Accessed: 2024-06-12. 2018.

[42] Haitao Liu et al. "Deep latent-variable kernel learning". In: *IEEE Transactions on Cybernetics* 52.10 (2021), pp. 10276–10289.

[43] Julien Mairal et al. "Convolutional kernel networks". In: *Advances in neural information processing systems* 27 (2014).

[44] Warren S McCulloch and Walter Pitts. "A logical calculus of the ideas immanent in nervous activity". In: *The bulletin of mathematical biophysics* 5 (1943), pp. 115–133.

[45] Siamak Mehrkanoon and Johan AK Suykens. "Deep hybrid neural-kernel networks using random Fourier features". In: *Neurocomputing* 298 (2018), pp. 46–54.

[46] Edward Milsom, Ben Anson, and Laurence Aitchison. "Convolutional deep kernel machines". In: *arXiv preprint arXiv:2309.09814* (2023).

[47] S. Moro, P. Rita, and P. Cortez. *Bank Marketing*. UCI Machine Learning Repository. DOI: https://doi.org/10.24432/C5K306. 2014.

[48] Kenta Nakai. *Ecoli*. UCI Machine Learning Repository. DOI: https://doi.org/10.24432/C5388M. 1996.

[49] Radford M Neal. *Bayesian learning for neural networks*. Vol. 118. Springer Science & Business Media, 2012.

[50] Adityanarayanan Radhakrishnan et al. "Mechanism for feature learning in neural networks and backpropagation-free machine learning models". In: *Science* 383.6690 (2024), pp. 1461–1467.

[51] Ali Rahimi and Benjamin Recht. "Random features for large-scale kernel machines". In: *Advances in neural information processing systems* 20 (2007).

[52] Byron Roe. *MiniBooNE particle identification*. UCI Machine Learning Repository. DOI: https://doi.org/10.24432/C5QC87. 2005.

[53] Frank Rosenblatt. "The perceptron: a probabilistic model for information storage and organization in the brain." In: *Psychological review* 65.6 (1958), p. 386.

[54] David E Rumelhart, Geoffrey E Hinton, and Ronald J Williams. "Learning representations by back-propagating errors". In: *nature* 323.6088 (1986), pp. 533–536.

[55] Sagar Sharma, Simone Sharma, and Anidhya Athaiya. "Activation functions in neural networks". In: *Towards Data Sci* 6.12 (2017), pp. 310–316.

[56] Maruan Al-Shedivat et al. "Learning scalable deep kernels with recurrent structure". In: *Journal of Machine Learning Research* 18.82 (2017), pp. 1–37.

[57] V. Sigillito et al. *Ionosphere*. UCI Machine Learning Repository. DOI: https://doi.org/10.24432/C 1989.

[58] David Slate. *Letter Recognition*. UCI Machine Learning Repository. DOI: https://doi.org/10.2443 1991.

[59] Vladimir Vapnik and Alexey Chervonenkis. "Theory of pattern recognition". In: (1974).

[60] Analytics Vidhya. *Insight into SVM (Support Vector Machine) along with code*. `https://www.analyticsvidhya.com/blog/2021/04/insight-into-svm-support-vector-machine-along-with-code/`. Apr. 2021.

[61] Jingyuan Wang, Kai Feng, and Junjie Wu. "SVM-based deep stacking networks". In: *Proceedings of the AAAI conference on artificial intelligence*. Vol. 33. 01. 2019, pp. 5273–5280.

[62] Daniel Whiteson. *HIGGS*. UCI Machine Learning Repository. DOI: https://doi.org/10.24432/C5V 2014.

[63] Marco A Wiering and Lambert RB Schomaker. "Multi-layer support vector machines". In: *Regularization, optimization, kernels, and support vector machines* 19 (2014), p. 457.

[64] Marco A Wiering et al. "The neural support vector machine". In: *BNAIC 2013: Proceedings of the 25th Benelux Conference on Artificial Intelligence, Delft, The Netherlands, November 7-8, 2013*. Delft University of Technology (TU Delft); under the auspices of the Benelux ... 2013.

[65] Andrew Gordon Wilson et al. "Deep kernel learning". In: *Artificial intelligence and statistics*. PMLR. 2016, pp. 370–378.

[66] Adam X Yang et al. "A theory of representation learning gives a deep generalisation of kernel methods". In: *International Conference on Machine Learning*. PMLR. 2023, pp. 39380–39415.

[67] Nicholas Young. *An introduction to Hilbert space*. Cambridge university press, 1988.

[68] Zhi-Hua Zhou and Ji Feng. "Deep forest". In: *National science review* 6.1 (2019), pp. 74–86.

[69] Matjaz Zwitter and Milan Soklic. *Breast Cancer*. UCI Machine Learning Repository. DOI: https://doi.org/10.24432/C51P4M. 1988.

A Overfitting capacity results

Model	P	Train accuracy	Test accuracy
radial			
Linear SVM	—	0.577 ± 0.006	0.482 ± 0.023
Decision tree (depth 3)	—	0.577 ± 0.014	0.491 ± 0.006
Arc-cosine	100	0.594 ± 0.005	0.489 ± 0.005
Arc-cosine	500	0.759 ± 0.007	0.493 ± 0.008
Arc-cosine	1000	1.000 ± 0.000	0.511 ± 0.016
Arc-cosine	2000	1.000 ± 0.000	0.499 ± 0.016
Arc-cosine	5000	1.000 ± 0.000	0.499 ± 0.008
RBF	100	0.611 ± 0.005	0.487 ± 0.013
RBF	500	0.763 ± 0.012	0.517 ± 0.034
RBF	1000	1.000 ± 0.000	0.509 ± 0.019
RBF	2000	1.000 ± 0.000	0.494 ± 0.003
RBF	5000	1.000 ± 0.000	0.493 ± 0.015
spiral			
Linear SVM	—	0.612 ± 0.020	0.576 ± 0.015
Decision tree (depth 3)	—	0.606 ± 0.008	0.569 ± 0.006
Arc-cosine	100	0.627 ± 0.008	0.529 ± 0.008
Arc-cosine	500	0.779 ± 0.005	0.531 ± 0.009
Arc-cosine	1000	1.000 ± 0.000	0.510 ± 0.019
Arc-cosine	2000	1.000 ± 0.000	0.531 ± 0.025
Arc-cosine	5000	1.000 ± 0.000	0.526 ± 0.006
RBF	100	0.629 ± 0.014	0.553 ± 0.014
RBF	500	0.780 ± 0.015	0.516 ± 0.010
RBF	1000	1.000 ± 0.000	0.533 ± 0.007
RBF	2000	1.000 ± 0.000	0.518 ± 0.011
RBF	5000	1.000 ± 0.000	0.511 ± 0.025
checker			
Linear SVM	—	0.570 ± 0.003	0.505 ± 0.017
Decision tree (depth 3)	—	0.560 ± 0.014	0.493 ± 0.009
Arc-cosine	100	0.613 ± 0.008	0.505 ± 0.011
Arc-cosine	500	0.768 ± 0.007	0.499 ± 0.018
Arc-cosine	1000	1.000 ± 0.000	0.521 ± 0.008
Arc-cosine	2000	1.000 ± 0.000	0.504 ± 0.005
Arc-cosine	5000	1.000 ± 0.000	0.517 ± 0.018
RBF	100	0.604 ± 0.003	0.505 ± 0.007
RBF	500	0.762 ± 0.008	0.513 ± 0.020
RBF	1000	1.000 ± 0.000	0.526 ± 0.021
RBF	2000	1.000 ± 0.000	0.515 ± 0.007
RBF	5000	1.000 ± 0.000	0.508 ± 0.006

Table 7: Experiment 1, full results. Training and test accuracy (mean ± standard deviation over three seeds) for the reference models and the proposed model with the arc-cosine and RBF kernels, across the full grid of random-feature dimensions P, on the three synthetic problems (*radial*, *spiral*, *checker*; each $n = 1500$, $d = 50$).

B Feature-Budget Control

Table 8 compares, at $m = 10$ and $m = 50$, the test accuracy obtained when each SVM is given the full feature budget $P = 1000$ against an earlier scheme allocating only P/m features per SVM. The full-P budget matches or exceeds the P/m budget at every comparable configuration, and the gap widens with m, since P/m shrinks the budget in proportion to the width. This confirms that the feature budget is a genuine confound, and justifies fixing it at the full P across all strategies in the main comparison.

Dataset	Subspace	m	full P	old P/m
CoverType	D (d/m)	10	0.7204 ± 0.0080	0.7253 ± 0.0272
CoverType	D (d/m)	50	0.7316 ± 0.0133	0.7275 ± 0.0317
CoverType	D ($\sqrt{d}$)	10	0.6892 ± 0.0286	0.6978 ± 0.0370
CoverType	D ($\sqrt{d}$)	50	0.7526 ± 0.0055	0.7428 ± 0.0328
FashionMNIST	D (d/m)	10	0.8383 ± 0.0370	
FashionMNIST	D ($\sqrt{d}$)	10	0.8182 ± 0.0373	
Gisette	D (d/m)	10	0.9437 ± 0.0076	
Gisette	D ($\sqrt{d}$)	10	0.9070 ± 0.0123	
HIGGS	D (d/m)	10	0.6296 ± 0.0185	0.6382 ± 0.0194
HIGGS	D ($\sqrt{d}$)	10	0.6146 ± 0.0254	0.6253 ± 0.0239
HIGGS	D ($\sqrt{d}$)	50	0.6544 ± 0.0177	0.6582 ± 0.0209
MNIST	D (d/m)	10	0.9186 ± 0.0127	0.9029 ± 0.0328
MNIST	D (d/m)	50	0.9062 ± 0.0058	0.8867 ± 0.0472
MNIST	D ($\sqrt{d}$)	10	0.8779 ± 0.0192	0.8707 ± 0.0457
MNIST	D ($\sqrt{d}$)	50	0.9186 ± 0.0050	0.8801 ± 0.0464
Optdigits	D (d/m)	10	0.9597 ± 0.0088	
Optdigits	D ($\sqrt{d}$)	10	0.9432 ± 0.0127	

Table 8: Feature-budget ablation ($m > 1$): old P/m vs full P per SVM for the data-split D arms. Quarantined from the feeding ranking.

C Per-Dataset Feeding Results

Tables 9–14 report the test accuracy (mean ± one seed standard deviation over three seeds) of every diversity strategy at $m = 10$, for each dataset and training size. A dash indicates a configuration not run because partitioning the n rows among the $m = 10$ SVMs would leave fewer than 200 samples per SVM. The best entry per column is shown in bold; differences within one standard deviation of the best are not statistically significant. These tables underlie the ranking of Table 3 and the regime split of Figure 10.

Arm	n=1k	n=5k	n=10k	n=50k	n=100k	n=200k
Feature Partitioning	**.5947 ± .0029**	.6253 ± .0046	.6376 ± .0058	.6544 ± .0027	.6551 ± .0033	.6535 ± .0033
Feature Subsampling	.5805 ± .0128	.6110 ± .0133	.6254 ± .0205	.6483 ± .0127	.6543 ± .0089	.6570 ± .0109
Instance Partitioning	—	**.6387 ± .0020**	**.6445 ± .0059**	**.6703 ± .0022**	**.6726 ± .0064**	**.6767 ± .0052**
Joint Part. (d/m)	—	.6052 ± .0205	.6244 ± .0087	.6427 ± .0035	.6455 ± .0029	.6484 ± .0042
Joint Part. ($\sqrt{d}$)	—	.5841 ± .0106	.6049 ± .0141	.6268 ± .0080	.6387 ± .0104	.6461 ± .0105

Table 9: Feeding strategies on HIGGS at $m = 10$ (mean ± std; best per column in bold). Differences within one standard deviation are not significant.

Arm	n=1k	n=5k	n=10k	n=50k
Feature Partitioning	**.6720 ± .0056**	.7286 ± .0026	.7570 ± .0042	—
Feature Subsampling	.6347 ± .0363	.6975 ± .0236	.7256 ± .0206	.7513 ± .0158
Instance Partitioning	—	**.7365 ± .0038**	**.7610 ± .0044**	**.7946 ± .0030**
Joint Part. (d/m)	—	.7178 ± .0033	.7302 ± .0038	—
Joint Part. ($\sqrt{d}$)	—	.6822 ± .0331	.7010 ± .0320	—

Table 10: Feeding strategies on CoverType at $m = 10$ (mean ± std; best per column in bold). Differences within one standard deviation are not significant.

Arm	n=500	n=1000	n=2000	n=3823
Feature Partitioning	**.9345 ± .0091**	**.9496 ± .0072**	.9647 ± .0032	.9739 ± .0037
Feature Subsampling	.9075 ± .0126	.9267 ± .0098	.9439 ± .0161	.9549 ± .0054
Instance Partitioning	—	—	**.9757 ± .0054**	**.9816 ± .0029**
Joint Part. (d/m)	—	—	.9615 ± .0018	.9659 ± .0014
Joint Part. ($\sqrt{d}$)	—	—	.9380 ± .0049	.9605 ± .0052

Table 11: Feeding strategies on Optdigits at $m = 10$ (mean ± std; best per column in bold). Differences within one standard deviation are not significant.

Arm	n=1000	n=5000	n=10000	n=50000
Feature Partitioning	**.8928 ± .0046**	**.9430 ± .0033**	**.9541 ± .0008**	**.9709 ± .0010**
Feature Subsampling	.8276 ± .0220	.8979 ± .0050	.9165 ± .0051	.9388 ± .0039
Instance Partitioning	—	.9134 ± .0019	.9282 ± .0038	.9490 ± .0006
Joint Part. (d/m)	—	.9086 ± .0013	.9316 ± .0023	—
Joint Part. ($\sqrt{d}$)	—	.8800 ± .0062	.8999 ± .0046	—

Table 12: Feeding strategies on MNIST at $m = 10$ (mean ± std; best per column in bold). Differences within one standard deviation are not significant.

Arm	n=2000	n=10000	n=50000
Feature Partitioning	**.8340 ± .0041**	**.8696 ± .0023**	**.8915 ± .0010**
Feature Subsampling	.8075 ± .0043	.8506 ± .0023	.8716 ± .0007
Instance Partitioning	.8112 ± .0010	.8554 ± .0022	.8758 ± .0023
Joint Part. (d/m)	.7909 ± .0037	.8535 ± .0034	.8763 ± .0002
Joint Part. ($\sqrt{d}$)	.7748 ± .0034	.8394 ± .0026	.8581 ± .0029

Table 13: Feeding strategies on Fashion-MNIST at $m = 10$ (mean ± std; best per column in bold). Differences within one standard deviation are not significant.

Arm	n=2000	n=5600
Feature Partitioning	**.9669 ± .0047**	**.9709 ± .0008**
Feature Subsampling	.9364 ± .0131	.9460 ± .0104
Instance Partitioning	.9210 ± .0030	.9298 ± .0037
Joint Part. (d/m)	.9376 ± .0039	.9507 ± .0007
Joint Part. ($\sqrt{d}$)	.9024 ± .0090	.9190 ± .0086

Table 14: Feeding strategies on Gisette at $m = 10$ (mean ± std; best per column in bold). Differences within one standard deviation are not significant.